AFPA

ECF 2
Gestion de Projet

BICHEL Anthony - GOTZELMANN John – VRENEZI Viktor

Groupe Projet 1
17/02/2026

ABSTRACT

- Auteurs : BICHEL Anthony - GOTZELMANN John – VRENEZI Viktor
- Correcteurs : BICHEL Anthony - GOTZELMANN John – VRENEZI Viktor
- Relecteurs : BICHEL Anthony - GOTZELMANN John – VRENEZI Viktor
- Modificateurs : BICHEL Anthony - GOTZELMANN John – VRENEZI Viktor
- Création :17/02/2026
- Date de dernière modification : 04/03/2026
- Approbateur : BICHEL Anthony
- Date d'approbation : 04/03/2026
- Version, Type, Status : Terminé Version 0.2.3
- ECF2, Destination Client AFPA Pompey

Tableau de Versionning

Date	Action	Auteur	Version
17/02/2026	Création	BICHEL Anthony	0.1.0
24/02/2026	Modification	BICHEL Anthony	0.1.1
25/02/2026	Modification	BICHEL Anthony	0.1.2
26/02/2026	Modification	VRENEZI Viktor	0.1.3
02/03/2026	Modification	VRENEZI Viktor	0.1.4
02/03/2026	Modification	BICHEl Anthony	0.2.1
04/03/2026	Modification	GOTZELMANN JOHN	0.2.2
04/03/2026	Modification	BICHEl Anthony	0.2.3
04/03/2026	Relecteur	VRENEZI Viktor	0.2.3
04/03/2026	Relecteur	GOTZELMANN JOHN	0.2.3
04/03/2026	Relecteur	BICHEL Anthony	0.2.3

Table des matières

I- Cahier des charges. 7
1. Objectif 7
2. Vue globale des composants 7
3. Schéma logique 8
4. Rôle de chaque brique (résumé) 8
5. Intégration dans ton TP1 (rappel rapide) 9
II- Matrice des exigences. 10
1. Exigences Fonctionnelles (EF) 10
2. Exigences Non fonctionnelles (NF) 11
3. Exigences d'intégration (INT) 12
III- Méthode. 12
1. Planification en Diagramme de Gantt avec MS Project. 13
2. Conception. 13
3. Déploiement 14
3.1Phase 1 — Infrastructure de base (VM POC VMWARE WORKSTATION) 14
3.2 Phase 2 — Authentification 15
3.3 Phase 3 — SIEM 15
3.4 Phase 4 — Supervision 15
3.5 Phase 5 — Automatisation & ticketing 15
3.6 Phase 6 — Intégration SSO 15
4. Exploitation 16
4.1 Surveillance quotidienne 16
4.2 Gestion des incidents 16
4.3 Accès aux services 16
IV- Schéma réseau. 16
1. Schéma réseau avec FossFlow. 16
2. Schéma réseau. 18
3. Plan d'adressage réseau. 19
3.1 VLANs 19
3.2 Équipements et Adresses IP 19
3.3 Flux Réseau 20
3.4 Routes Retour sur le Fortinet. 21
3.5 Polices Firewall Fortinet. 21
4. Schéma des flux réseau. 22
V- CMDB — Gestion des Actifs 23

1. Inventaire Matériel Physique 23
2. Actifs Logiciels & Infrastructure Virtuelle 24
3. Actifs Données & Interfaces 25
4. Matrice de Criticité & Plan de Continuité 26
5. Actifs Humains & Parties Prenantes 26
6. Légende des Criticités 27
VI – Mise en place de la VM POC 27
1 Infrastructure de Base 27
1.1 Machine Hôte 27
1.2 Prérequis Installés 27
1.3 Structure des Répertoires 28
2. Services Déployés 28
2.1 : Configuration des services Docker : 28
2.1.1 Structure des Dossiers 28
2.1.2 Wazuh Stack 29
2.1.3. Portainer 32
2.2 Portainer — Gestion Docker 33
2.3 Stack Wazuh — SIEM Principal 33
2.3.1wazuh-indexer (OpenSearch) 33
2.3.2wazuh-manager 34
2.3.3 wazuh-dashboard (OpenSearch Dashboards) 34
2.4 Nginx — Reverse Proxy 34
3. Configuration Réseau 35
3.1 Réseau Docker 35
3.2 Ports Exposés sur l'Hôte 35
4. Credentials Configurés 36
5. Problèmes Rencontrés et Solutions 36
Problème 1 — vm.max_map_count trop bas 36
Problème 2 — Certificats SSL manquants 36
Problème 3 — Filebeat SSL verification_mode: full 37
Problème 4 — Filebeat s'arrêtait après pkill 37
Problème 5 — Mémoire insuffisante 37
Problème 6 — Index pattern déjà existant 37
6. Flux de Données 37
7. État Final Vérifié 38
8. Commandes Utiles de Maintenance 41

8.1 Supervision 41
8.2 Redémarrage 41
8.3 Alertes en Temps Réel 41
8.4 Santé du Cluster 41
9. Intégration Wazuh → Shuffle → GLPI 42
9.1 Architecture du Workflow 42
9.2 Configuration Wazuh (ossec.conf) 42
9.3 Workflow Shuffle — Détail des étapes 43
10. Supervision Avancée — Stack Monitoring (Docker) 44
10.1 Architecture de la Stack 44
10.2 Configuration Prometheus 44
10.3 Connexion Prometheus → Grafana 45
10.4 Connexion Prometheus → Alertmanager 46
10.5 Résultats Obtenus 47
VII - Glossaire 51
VIII - Conclusion 53

I- Cahier des charges.

Projet 1 — supervision++ infra AIS

1. Objectif

Infrastructure AIS ou "TPE / petit environnement"

Une seule VM Debian 12 au minimum, ajout d'une LXC Debian 12 possible

Tout en Docker autant que possible

Supervision, détection basique, SSO, alertes mail

Déploiement automatisable (Proxmox + Ansible + Python)

2. Vue globale des composants

VM unique : cyberbox-light (Debian 12)

Docker + Docker Compose + Portainer

Wazuh Manager (ou Wazuh central pour quelques agents)

LibreNMS (supervision réseau)

Loki (+ promtail sur la box, éventuellement sur d'autres machines)

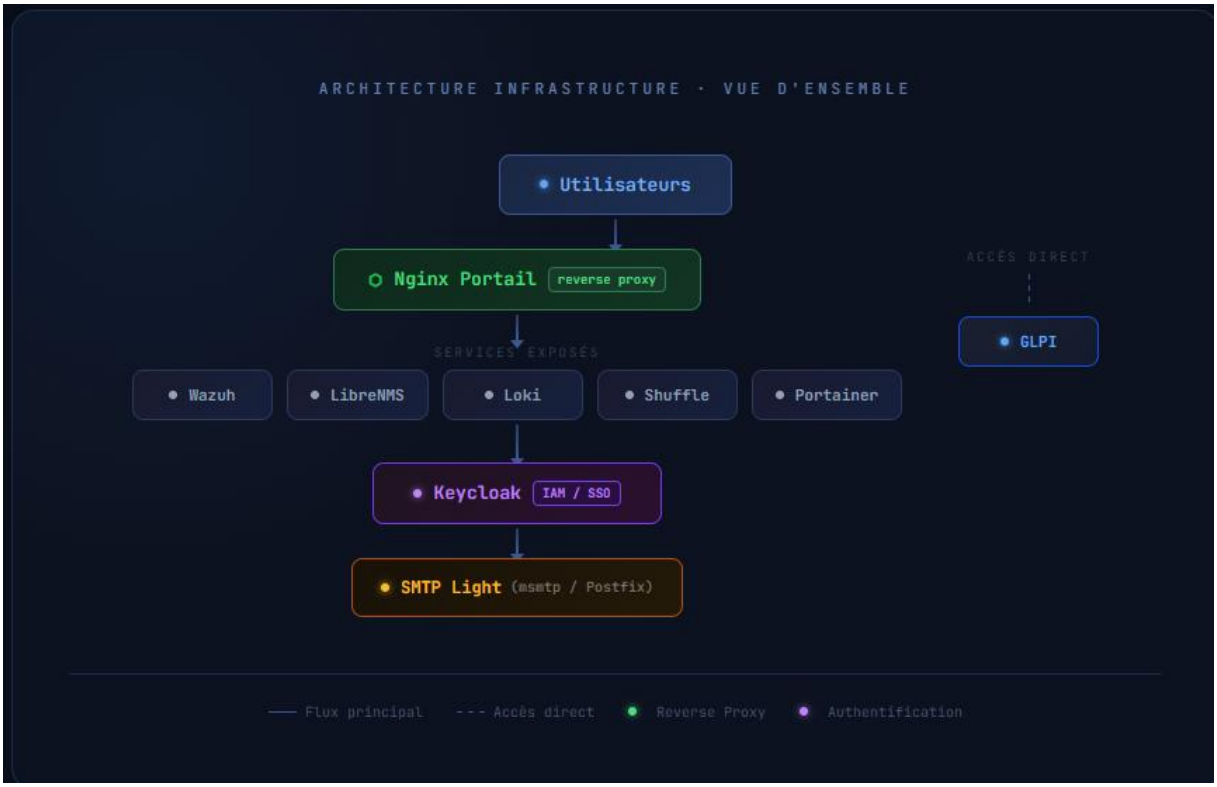
Shuffle (SOAR léger, pour orchestrer quelques workflows)

Keycloak (SSO + MFA pour les services web)

SMTP léger (msmtp ou Postfix relay-only, pour l'envoi d'e-mails)

Nginx portail (page web unique listant les services, ou simple reverse proxy)

3. Schéma logique



Les utilisateurs accèdent à une page portail Nginx (ou directement aux services).

Les services web (Wazuh, LibreNMS, Portainer, GLPI) doivent être protégés par Keycloak (OIDC).

Tous les services qui envoient des alertes (Wazuh, LibreNMS, Shuffle, Loki via alertmanager si tu ajoutes Prometheus) utilisent le SMTP léger.

4. Rôle de chaque brique (résumé)

Wazuh Manager : détection basique sur les postes, centralisation de logs agents, quelques règles de sécurité.

LibreNMS : supervision réseau (switchs, routeurs, AP, etc.).

Loki : centralisation de logs (système, applicatifs, conteneurs).

Shuffle : automatisation simple (ex : si alerte Wazuh critique → envoi mail + création ticket).

Keycloak : SSO + MFA pour les interfaces web (Wazuh, LibreNMS, Portainer…).

SMTP léger : envoi d'e-mails d'alerte (sans boîte mail locale).

Nginx portail : point d'entrée unique, ergonomique.

5. Intégration dans ton TP1 (rappel rapide)

Proxmox : une VM Debian 12 clonable (template).

Ansible : rôles docker, wazuh_manager, librenms, loki, shuffle, keycloak, smtp_light, nginx_portal, portainer.

Python : script qui appelle l'API Proxmox pour cloner la VM, puis lance les playbooks Ansible.

II- Matrice des exigences.

1. Exigences Fonctionnelles (EF)

Ce sont les fonctions ou services quel système doit fournir. Elles décrivent ce que le système doit faire pour répondre aux besoins des utilisateurs ou des processus métiers.

ID	Exigence client	Solution Technique	BUT :
EF1	Détecter les intrusions et centraliser les logs	Wazuh Manager (Docker) : Déploiement avec règles de détection basiques et agents Wazuh sur les postes clients.	Les logs des agents sont visibles dans l'interface Wazuh et une alerte de test est déclenchée.
EF2	Superviser les équipements réseau	LibreNMS (Docker) : Ajout des switchs, routeurs et AP via SNMP.	Les équipements sont listés dans LibreNMS et leur statut est monitoré (up/down, trafic).
EF3	Centraliser les logs système et applicatifs	Loki + Promtail (Docker) : Collecte des logs de la VM, des conteneurs et des services.	Les logs sont accessibles via Grafana ou l'API Loki (ex : journalctl redirigé vers Loki).
EF4	Automatiser les workflows d'alerte	Shuffle (Docker) : Workflow "Alerte critique → Envoi mail + Création ticket GLPI".	Un workflow de test déclenche un email et crée un ticket dans GLPI.
EF5	Authentification unique (SSO + MFA)	Keycloak (Docker) : Intégration OIDC avec Nginx pour Wazuh, LibreNMS, Portainer.	Connexion réussie via Keycloak sur tous les services web (test avec un utilisateur de test).
EF6	Envoyer des alertes par email	SMTP léger (msmtp/Postfix relay-only, Docker) : Configuration comme relais pour Wazuh, LibreNMS, Shuffle.	Test d'envoi d'email depuis chaque service (ex : alerte Wazuh → email reçu).
EF7	Accès unifié aux services	Nginx (Docker) : Reverse proxy + portail web listant les services (Wazuh, LibreNMS, etc.).	Tous les services sont accessibles via https://cyberbox-light/portail sans erreur 404.
EF8	Gestion centralisée des conteneurs Docker	Portainer (Docker) : Déploiement pour gérer les stacks Docker.	Portainer affiche tous les conteneurs et permet de les redémarrer/stopper.

2. Exigences Non fonctionnelles (NF).

Ce sont les fonctions ou services que le système doit fournir. Elles décrivent ce que le système doit faire pour répondre aux besoins des utilisateurs ou des processus métiers.

ID	Exigence	Solution Technique	Validation
NF1	Déploiement 100% automatisable	Proxmox (template VM Debian 12) + Ansible (rôles pour chaque service) + Script Python (API Proxmox).	La VM est clonée et tous les services sont déployés via ansible-playbook deploy.yml.
NF2	Tout en Docker	Docker Compose : Fichiers docker-compose.yml pour chaque service (Wazuh, LibreNMS, etc.).	docker ps liste tous les conteneurs attendus (ex : wazuh-manager, librenms).
NF3	Environnement "TPE/petit" (1 VM + 1 LXC max)	1 VM Debian 12 (cyberbox-light) + 1 LXC optionnelle pour isoler un service (ex : GLPI).	La VM et le LXC (si utilisé) sont opérationnels et accessibles.
NF4	Alertes configurables et personnalisables	Wazuh (règles personnalisées) + LibreNMS (seuils) + Loki (Alertmanager si Prometheus ajouté).	Une alerte de test (ex : CPU > 90%) est déclenchée et reçue par email.
NF5	Sécurité basique (MFA, isolation conteneurs)	Keycloak (MFA obligatoire) + Docker (réseaux isolés, utilisateurs non-root, read-only fs).	MFA requis pour accéder aux services ; conteneurs en mode non-root (docker inspect).

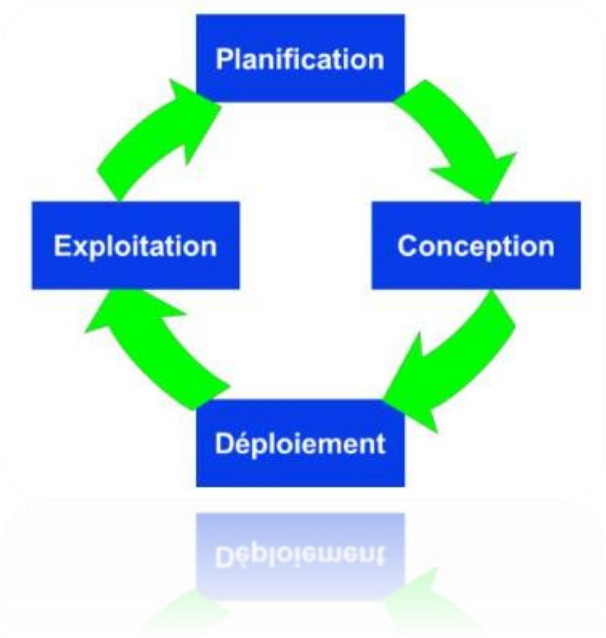
3. Exigences d'intégration (INT)

Ce sont les interactions entre les composants du système ou avec des systèmes externes. Elles garantissent que les différentes briques communiquent correctement.

ID	Exigence	Solution Technique	Validation
INT1	Wazuh → SMTP (envoi d'alertes)	Configuration du relais SMTP dans Wazuh (ossec.conf →).	Une alerte Wazuh génère un email reçu dans la boîte de test.
INT2	LibreNMS → SMTP (envoi d'alertes)	Configuration du relais SMTP dans LibreNMS (config.php → $config['email_backend']).	Une alerte réseau (ex : lien down) génère un email.
INT3	Shuffle → GLPI (création de tickets)	App Shuffle "GLPI Connector" : API GLPI configurée pour créer des tickets.	Un workflow Shuffle crée un ticket visible dans GLPI.
INT4	Keycloak → Nginx (protection des services)	Nginx : Module auth_request + Keycloak (client OIDC pour chaque service).	Accès refusé à Wazuh/LibreNMS sans authentification Keycloak.
INT5	Loki → Grafana (visualisation des logs)	Grafana (Docker) : Source de données Loki + dashboard "Logs".	Les logs sont visibles et filtrables dans Grafana.

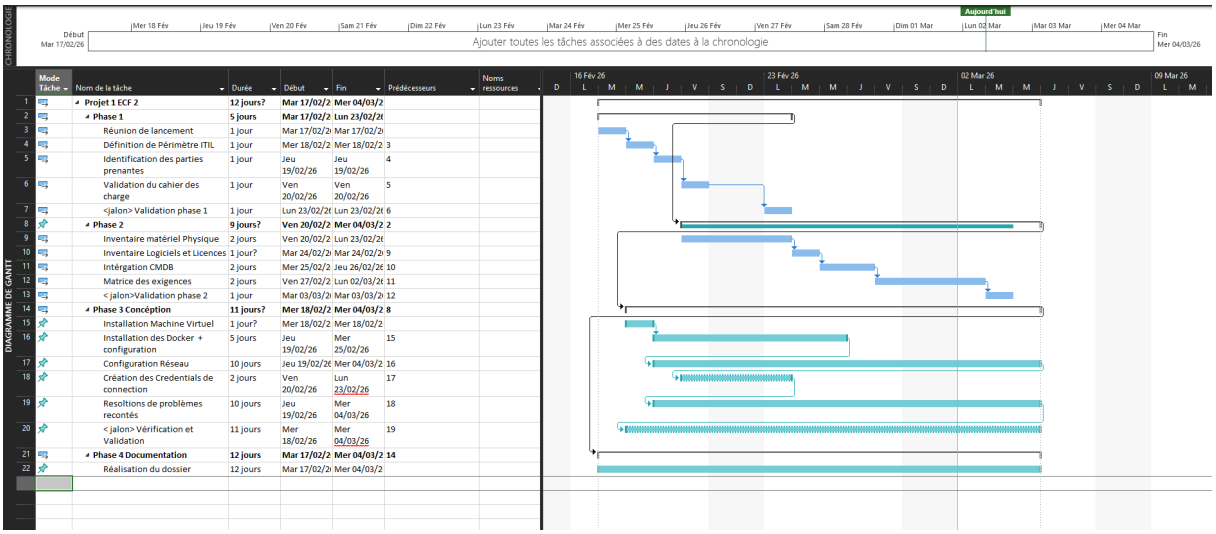
III- Méthode.

La méthode utilisée est le cycle de vie ITIL.



1. Planification en Diagramme de Gantt avec MS Project.

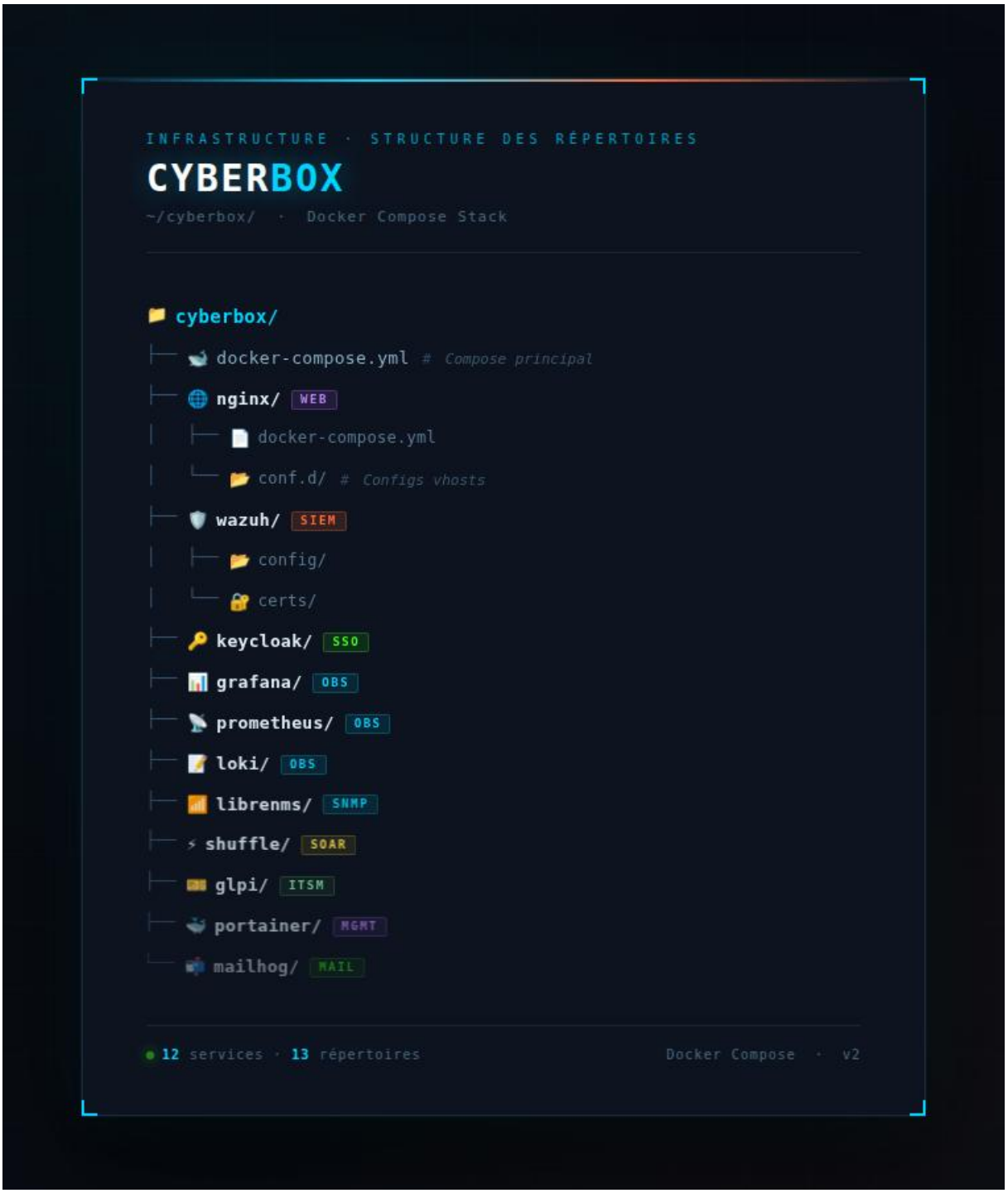
(Voir le document en format A4 en Annexe 1)



2. Conception.

Déploiement d'une infrastructure de supervision complète pour un petit environnement (TPE/AIS) sur une VM Debian 12 unique, entièrement conteneurisée avec Docker.

Composant	Rôle	Port(s)
Nginx	Portail / Reverse Proxy HTTPS	80, 443
Keycloak	SSO — Authentification centralisée	8080
Wazuh Manager	SIEM — Collecte & analyse logs	1514, 1515, 55000
Wazuh Indexer	Base OpenSearch — Stockage alertes	9200
Wazuh Dashboard	Interface visualisation Wazuh	5601
LibreNMS	Supervision réseau SNMP	8000
Prometheus	Collecte métriques	9090
Grafana	Dashboards métriques	3000
Loki	Agrégation de logs	3100
Shuffle	SOAR — Automatisation réponse incidents	3001
GLPI	Gestion tickets / CMDB	8008
Portainer	Interface gestion Docker	9000
Mailhog / Postfix	Alertes mail	1025, 8025



3. Déploiement

3.1Phase 1 — Infrastructure de base (VM POC VMWARE WORKSTATION)

1. Docker + réseau cyberbox

2. Portainer (gestion)

3. Nginx (portail)

3.2 Phase 2 — Authentification

4. Keycloak (SSO) est une solution open source de gestion des identités et des accès. Elle permet aux utilisateurs de se connecter à des applications avec un seul compte. Elle propose l'authentification unique (Dingle Sign-On), la gestion de sessions et la fédération d'indentité.

3.3 Phase 3 — SIEM

5. Wazuh Indexer

6. Wazuh Manager

7. Wazuh Dashboard

3.4 Phase 4 — Supervision

8. Prometheus + Loki

9. Grafana 10. LibreNMS

3.5 Phase 5 — Automatisation & ticketing

11. Shuffle (SOAR)

12. GLPI

13. Mailhog / Postfix

3.6 Phase 6 — Intégration SSO

14. Connexion Keycloak → tous les services

4. Exploitation.

4.1 Surveillance quotidienne

- Consultation des dashboards **Grafana** (métriques CPU, RAM, réseau)
- Vérification des alertes **Wazuh** (tentatives d'intrusion, anomalies)

4.2 Gestion des incidents

- Détection automatique → **Wazuh** génère une alerte
- Déclenchement automatique → **Shuffle** reçoit l'alerte (webhook)
- Création automatique d'un ticket → **GLPI**
- Notification par mail → **Mailhog**

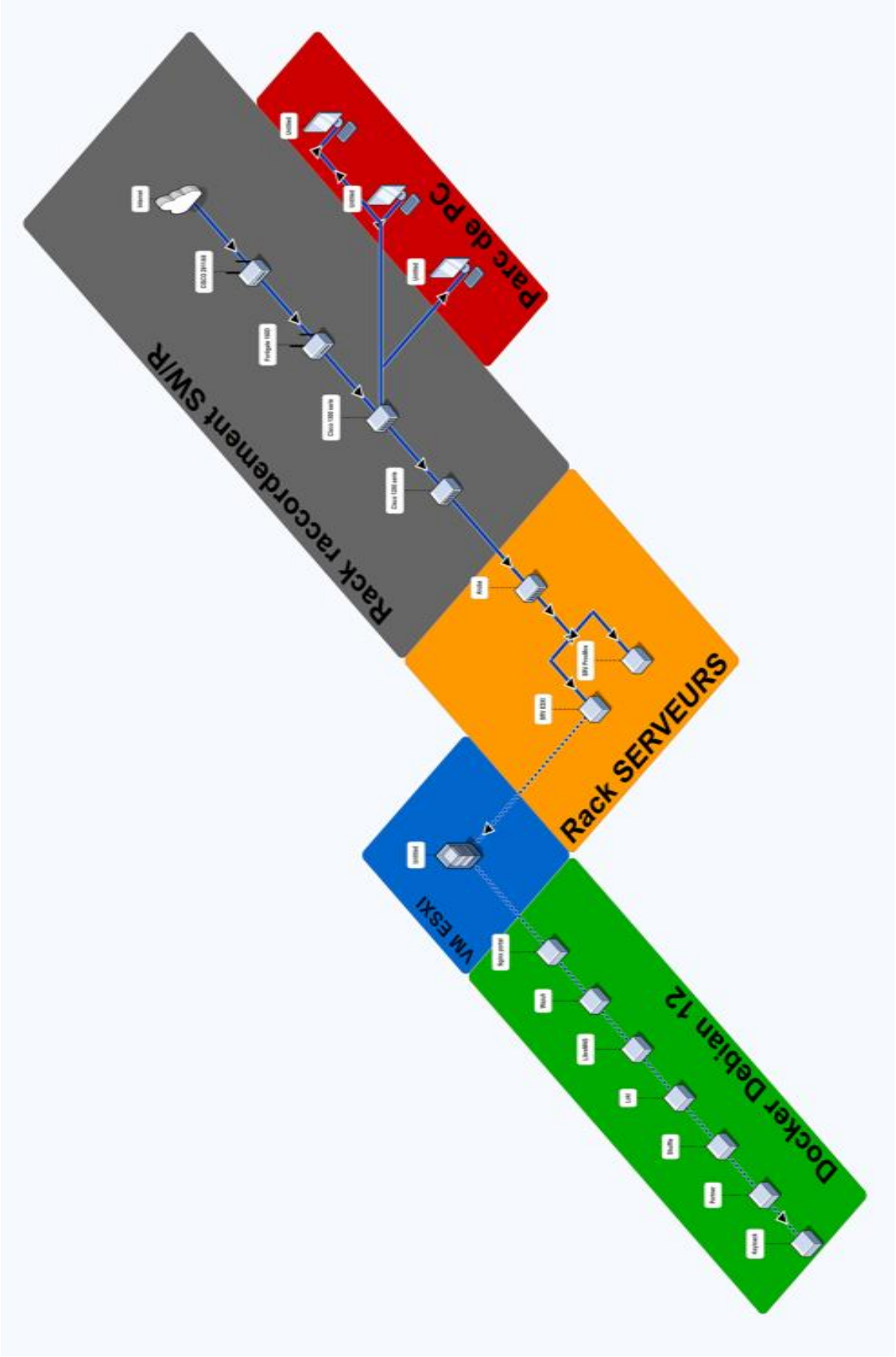
4.3 Accès aux services

- Authentification unique via **Keycloak** (SSO)
- Accès centralisé via le portail **Nginx** (HTTPS)
- Gestion des conteneurs via **Portainer**

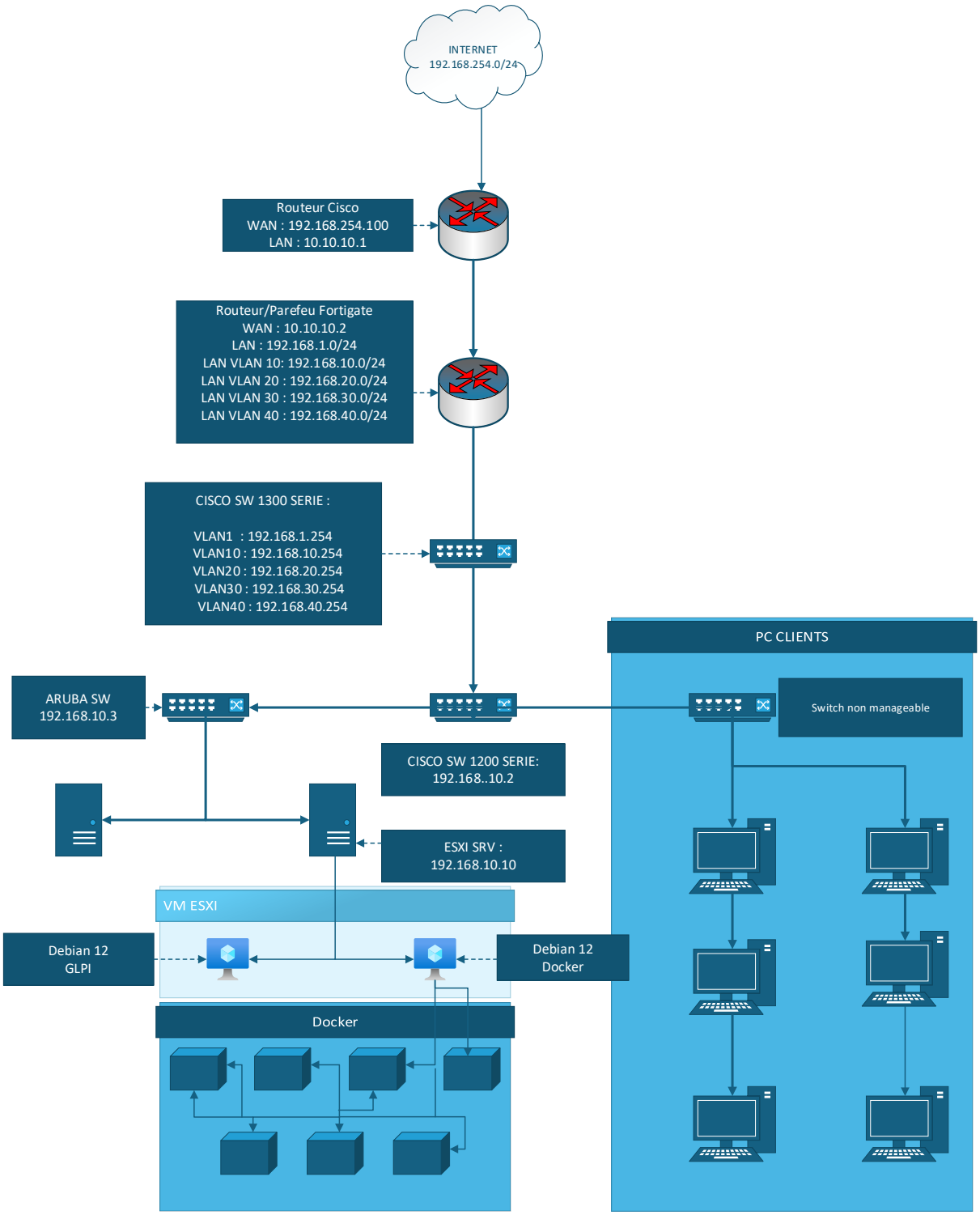
Se référer à la partie VI mise en place du POC pour de plus ample explication sur la mise en place de l'infra

IV- Schéma réseau.

1. Schéma réseau avec FossFlow.


Parc de PC
Rack raccordement SW/R
Rack SERVEURS
VM ESXI
Docker Debian 12

2. Schéma réseau.



3. Plan d'adressage réseau.

3.1 VLANs

1. VLANs			
VLAN	**Nom**	**Réseau**	**Description**
VLAN 1	TRANSIT	192.168.1.0/24	Lien N3 ↔ Fortinet
VLAN 10	MANAGEMENT	192.168.10.0/24	Administration réseau
VLAN 20	ESXI	192.168.20.0/24	Hyperviseur VMware
VLAN 30	PROXMOX	192.168.30.0/24	Hyperviseur PVE
VLAN 40	CLIENTS	192.168.40.0/24	Postes utilisateurs

3.2 Équipements et Adresses IP

2. Équipements et Adresses IP	
Routeur Cisco	
Interface / Service	**Adresse IP**
WAN	192.168.254.100
LAN	10.10.10.1

Fortinet (Firewall/NAT)	
Interface / Service	**Adresse IP**
WAN	10.10.10.2
LAN	192.168.1.254
DHCP	192.168.1.1 → 192.168.1.253

Switch N3 (Cœur de réseau)	
Interface / Service	**Adresse IP**
VLAN 1 (Transit)	192.168.1.1
VLAN 10 (Management)	192.168.10.1
VLAN 20 (ESXi)	192.168.20.1
VLAN 30 (Proxmox)	192.168.30.1
VLAN 40 (Clients)	192.168.40.1
Gateway par défaut	0.0.0.0/0 via 192.168.1.254

Switchs de Distribution / Accès	
Interface / Service	**Adresse IP**
Switch N2 (Distribution)	192.168.10.2
Switch Aruba (Accès serveurs)	192.168.10.3

Serveur ESXi	
Interface / Service	**Adresse IP**
Management (ESXi)	192.168.10.10
Interface VLAN 20	192.168.20.10
Nginx Portail	192.168.20.20
Wazuh (SIEM)	192.168.20.21
LibreNMS (Supervision)	192.168.20.22
Loki (Logs)	192.168.20.23
Shuffle (SOAR)	192.168.20.24
Portainer	192.168.20.25
GLPI (ITSM)	192.168.20.26
Keycloak (SSO)	192.168.20.27
SMTP (msmtp)	192.168.20.28

Serveur Proxmox	
Interface / Service	**Adresse IP**
Management	192.168.10.11
Interface VLAN 30	192.168.30.10

Clients (VLAN 40)	
Interface / Service	**Adresse IP**
Plage DHCP	192.168.40.100 → 192.168.40.200
Gateway	192.168.40.1

3.3 Flux Réseau

3. Flux Réseau			
Source	**Destination**	**Protocole / Port**	**Usage**
Clients	Nginx Portail	HTTPS 443	Accès portail
Nginx	Services	HTTP 80xx	Reverse proxy
Services	Keycloak	HTTPS 8443	Authentification
Keycloak	SMTP	TCP 587	Notifications mail
LibreNMS	Équipements	SNMP 161	Supervision réseau
Wazuh	Agents	TCP 1514	Collecte logs
Loki	Services	TCP 3100	Agrégation logs
Shuffle	Services	HTTP API	Automatisation SOAR

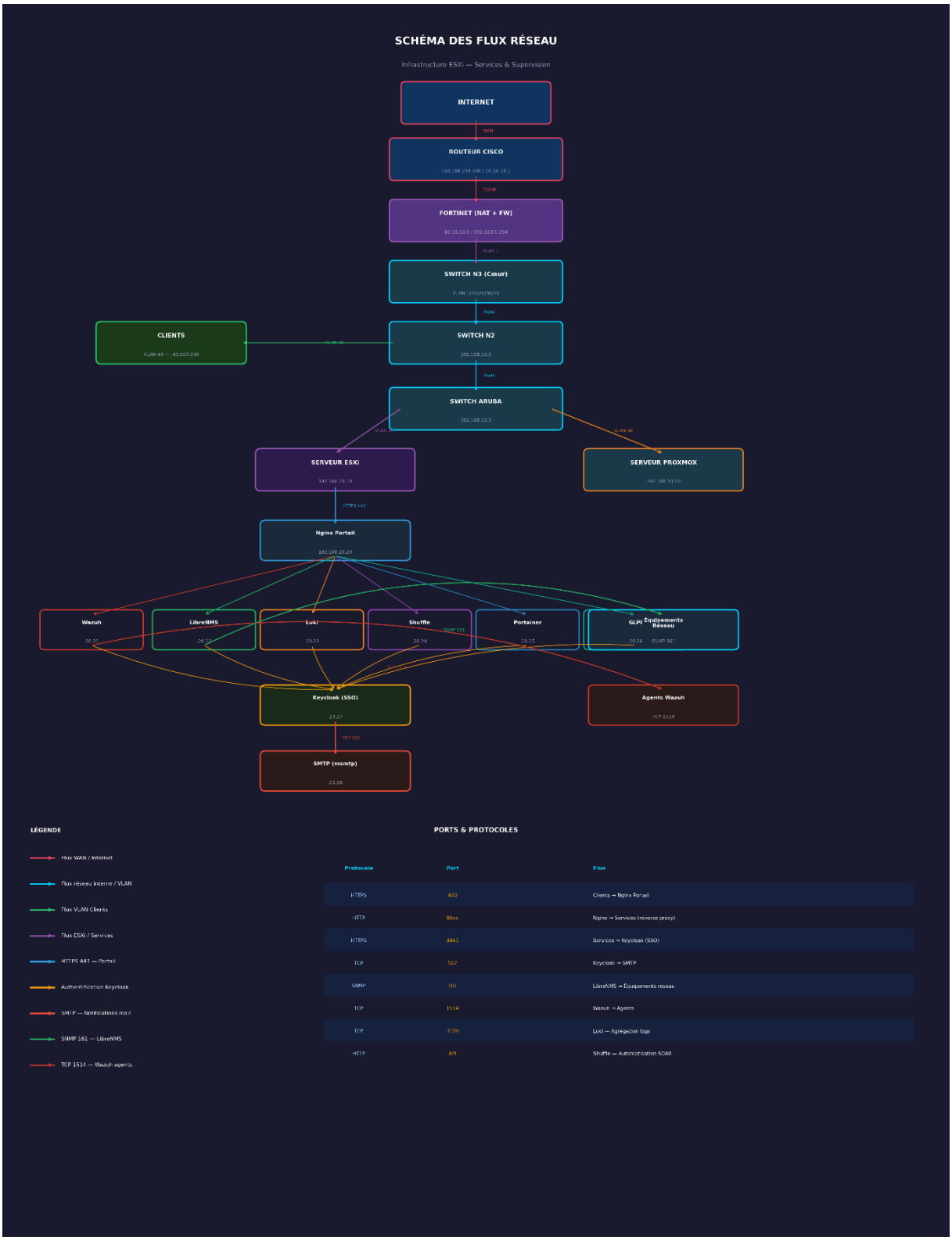
3.4 Routes Retour sur le Fortinet.

4. Routes Retour sur le Fortinet	
Réseau de destination	**Via (Next-hop)**
192.168.10.0/24 (Management)	192.168.1.1
192.168.20.0/24 (ESXi)	192.168.1.1
192.168.30.0/24 (Proxmox)	192.168.1.1
192.168.40.0/24 (Clients)	192.168.1.1

3.5 Polices Firewall Fortinet.

5. Policies Firewall Fortinet			
Source	**Destination**	**Action**	**Détail**
LAN 192.168.1.0/24	WAN	ALLOW + NAT	
VLAN 20 (ESXi)	WAN	ALLOW + NAT	
VLAN 30 (Proxmox)	WAN	ALLOW + NAT	
VLAN 40 (Clients)	WAN	ALLOW + NAT	
VLAN 40 (Clients)	VLAN 20 (Services)	ALLOW	HTTPS 443 uniquement
VLAN 10 (Management)	ALL	ALLOW	Accès total

4. Schéma des flux réseau.



V- CMDB — Gestion des Actifs

(Projet Supervision++ Infra AIS — Approche ITIL)

1. Inventaire Matériel Physique

Équipements physiques constituant l'infrastructure réseau et serveur sur site. Ces actifs forment le socle sur lequel repose l'ensemble des services logiciels du projet AIS.

ID	Marque	Modèle	Catégorie	Rôle dans l'infra AIS	Criticité	Statut
M-001	CISCO	Routeur 2911/K9	Matériel / Réseau	Routage principal, accès Internet, interconnexion sites	Critique	Actif
M-002	FORTINET	Fortigate 100D	Matériel / Sécurité	Pare-feu périmétrique, filtrage, VPN télétravailleurs	Critique	Actif
M-003	CISCO	Switch 1300 Séries	Matériel / Réseau	Commutation niveau distribution, VLAN	Élevé	Actif
M-004	CISCO	Switch 1200 Séries	Matériel / Réseau	Commutation niveau accès, connexion postes utilisateurs	Élevé	Actif
M-005	ARUBA	Instant On 1930 Séries	Matériel / Réseau	Switch manageable, WiFi guest, accès télétravailleurs	Élevé	Actif
M-006	DELL	Serveur PowerEdge R620	Matériel / Serveur	Hébergement Esxi, VM cyberbox-light, conteneurs Docker	Critique	Actif
M-007	ACER	AR 360 F2	Matériel / Serveur	Serveur secondaire ou poste de supervision	Moyen	Actif

2. Actifs Logiciels & Infrastructure Virtuelle

Composants logiciels déployés sur le serveur Dell PowerEdge R620 via Esxi. Chaque brique est conteneurisée sous Docker autant que possible.

ID	Catégorie	Sous-catégorie	Actif	Rôle ITIL	Dépend de	Criticité	Statut
A-001	Infrastructure	Virtualisation	Esxi	Hyperviseur, gestion VM/LXC	M-006 (Dell)	Critique	Actif
A-002	Infrastructure	VM	cyberbox-light (Deb12)	VM hôte de tous les services	A-001	Critique	Actif
A-003	Infrastructure	Conteneur optionnel	LXC Debian 12	Isolation de services secondaires	A-001	Moyen	Optionnel
A-004	Logiciel	Conteneurisation	Docker + Docker Compose	Orchestration des conteneurs	A-002	Critique	Actif
A-005	Logiciel	Orchestration	Portainer	Interface gestion Docker	A-004	Élevé	Actif
A-006	Logiciel	SIEM / EDR	Wazuh Manager	Détection menaces, centralisation logs	A-004, A-008	Critique	Actif
A-007	Logiciel	Supervision réseau	LibreNMS	Supervision switchs, routeurs, AP	A-004, M-001 à M-005	Élevé	Actif
A-008	Logiciel	Centralisation logs	Loki + Promtail	Agrégation logs système et conteneurs	A-004	Élevé	Actif
A-009	Logiciel	SOAR	Shuffle	Automatisation workflows sécurité	A-004, A-006, A-010	Moyen	Actif
A-010	Logiciel	SSO / MFA	Keycloak	Authentification centralisée OIDC	A-004	Critique	Actif
A-011	Logiciel	Messagerie	SMTP (msmtp/Postfix)	Envoi alertes mail	A-004	Moyen	Actif
A-012	Logiciel	Reverse Proxy	Nginx Portail	Point d'entrée unique, portail web	A-004, A-010	Élevé	Actif
A-013	Logiciel	ITSM	GLPI	Ticketing, gestion parc ITIL	A-004	Élevé	Actif
A-014	Logiciel	Automatisation	Ansible	Déploiement et configuration	A-001, A-002	Élevé	Actif
A-015	Logiciel	Scripting	Python (API Esxi)	Clone VM, lancement playbooks	A-001, A-014	Élevé	Actif

3. Actifs Données & Interfaces

ID	Catégorie	Actif / Flux	Description	Produit par	Criticité	Protection
D-001	Données	Logs Wazuh	Logs sécurité agents et règles SIEM	A-006	Critique	Volume Docker persistant, accès restreint
D-002	Données	Logs Loki	Logs système, applicatifs, conteneurs	A-008	Élevé	Volume Docker persistant
D-003	Données	Métriques LibreNMS	Données supervision réseau (SNMP, ping)	A-007	Élevé	BDD locale chiffrée
D-004	Données	Base Keycloak	Identités, tokens, sessions SSO	A-010	Critique	Volume chiffré, backup régulier
D-005	Données	Config Ansible	Playbooks, inventaires, variables chiffrées	A-014	Élevé	Ansible Vault, dépôt Git privé
D-006	Données	Tickets GLPI	Incidents, demandes, parc matériel CMDB	A-013	Élevé	Backup BDD quotidien
D-007	Interface	Portail Nginx (OIDC)	Accès web unique protégé par Keycloak	A-012, A-010	Élevé	HTTPS, certificat TLS
D-008	Interface	API Esxi	Interface de clonage VM via Python	A-001, A-015	Élevé	Token API, accès réseau restreint
D-009	Interface	Flux SMTP alertes	Envoi mails Wazuh, LibreNMS, Shuffle	A-011	Moyen	Relay sécurisé, authentification SMTP
D-010	Interface	Flux SNMP LibreNMS	Collecte métriques réseau (M-001 à M-005)	A-007	Élevé	SNMPv3, communauté restreinte

4. Matrice de Criticité & Plan de Continuité

ID	Actif	Impact si indisponible	Mitigation / Continuité
M-001	CISCO Routeur 2911/K9	Perte accès Internet total → déconnexion tous services cloud/mail/web	Lien 4G de secours, configuration sauvegardée
M-002	FORTINET Fortigate 100D	Perte sécurité périmétrique, VPN télétravailleurs coupé	Config exportée, matériel de remplacement référencé
M-006	DELL PowerEdge R620	Perte totale de tous les services AIS (Esxi + Docker)	Snapshot Esxi quotidien, procédure de restauration
A-001	Proxmox	Impossibilité de démarrer les VM/conteneurs	Template VM clonable, script Python de redéploiement
A-006	Wazuh Manager	Angle mort sécurité total, aucune détection d'incidents	Healthcheck Docker, redémarrage automatique
A-010	Keycloak	Perte d'accès à tous les services web (SSO cassé)	Volume persistant, backup config, procédure bypass
A-012	Nginx Portail	Perte du point d'entrée unique	Accès direct aux services en cas d'urgence
A-007	LibreNMS	Perte supervision réseau, incidents non détectés	Redémarrage auto Docker, alertes SMTP
A-008	Loki + Promtail	Perte centralisation logs, investigation impossible	Volume persistant, rotation des logs
D-004	Base Keycloak	Perte identités et sessions SSO	Backup chiffré quotidien, restauration testée

5. Actifs Humains & Parties Prenantes

Recensement des ressources humaines impliquées dans le projet Supervision++ AIS. Ces actifs sont enregistrés comme Configuration Items (CI) de type People conformément aux bonnes pratiques ITIL v4.

ID	Catégorie	Sous-catégorie	Actif (Personne)	Rôle ITIL	Dépend de	Criticité	Statut
PP-001	Humain	Gestion de projet	**Viktor**	Présentation des objectifs du projet, pilotage ITIL	PP-002, PP-003	**Critique**	**Actif**
PP-002	Humain	Méthode & Process	**Anthony**	Présentation de la mise en forme ITIL (méthodes, outils)	PP-001	**Critique**	**Actif**
PP-003	Humain	Technique	**John**	Présentation de l'aspect technique de l'infrastructure	PP-001, PP-002	**Critique**	**Actif**
PP-004	Humain	Prêt Matériel	**Alex**	Prêt du matériel nécessaire au projet (CI-001)	PP-001	**Moyen**	**Actif**
PP-005	Humain	Client / Commanditaire	**Vincent**	Client du projet, validation des livrables et objectifs	PP-001	**Critique**	**Actif**

Note ITIL : Les entrées PP-001 à PP-005 sont enregistrées comme Configuration Items (CI) de type People conformément aux bonnes pratiques ITIL v4. Le CI-001 (matériel prêté par Alex) devra être mis à jour en statut "Restitué" à la clôture du projet. L'ensemble de ces actifs est à intégrer dans GLPI (module Parc / Prêts matériels) pour une traçabilité complète des changements.

6. Légende des Criticités

Niveau	Code couleur	Signification
Critique	#C00000	Arrêt total du service si indisponible. Priorité absolue de restauration.
Élevé	#E26B0A	Impact majeur sur la production. Restauration sous 4h.
Moyen	#F4D03F	Impact partiel. Restauration sous 24h.
Faible	#70AD47	Impact limité. Restauration sous 72h.
Optionnel	#A9A9A9	Composant non obligatoire. Peut être ignoré sans impact critique.

Recommandation ITIL : Intégrer cette CMDB dans GLPI (module Parc) pour une traçabilité complète des changements sur chaque actif (CI — Configuration Item).

VI – Mise en place de la VM POC

1 Infrastructure de Base

1.1 Machine Hôte

Paramètre	Valeur
Système d'exploitation	Debian
Utilisateur principal	seiko (avec sudo)
Répertoire principal	/root/cyberbox/

1.2 Prérequis Installés

- Docker + Docker Compose
- curl, wget, git
- python3
- net-tools, htop

1.3 Structure des Répertoires

```
/root/cyberbox/
├── wazuh/
│   └── docker-compose.yml
├── portainer/
│   └── docker-compose.yml
├── nginx/
│   ├── docker-compose.yml
│   └── conf.d/
│       └── cyberbox.conf
└── [autres services]
```

2. Services Déployés

2.1 : Configuration des services Docker :

2.1.1 Structure des Dossiers

Le projet CyberBox est organisé selon l'arborescence suivante :

```
/root/cyberbox/
├── wazuh/
│   └── docker-compose.yml
├── portainer/
│   └── docker-compose.yml
├── misp/
│   └── docker-compose.yml
└── nginx/
    ├── docker-compose.yml
    └── nginx.conf
```

2.1.2 Wazuh Stack

Fichier : ~/cyberbox/wazuh/docker-compose.yml

La stack Wazuh comprend trois composants principaux :

wazuh-indexer

Moteur de stockage basé sur OpenSearch. Expose le port 9200 pour les requêtes internes.

wazuh-manager

Cœur du SIEM. Gère la réception des événements agents (ports 1514/1515) et l'API REST (port 55000).

wazuh-dashboard

Interface web accessible en HTTPS sur le port 443, reliée à l'indexer et au manager.

```
version: '3.8'


services:


  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.3
    container_name: wazuh-indexer
    restart: always
    ports:
      - "9200:9200"
    environment:
      - "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"
      - OPENSEARCH_INITIAL_ADMIN_PASSWORD=SecretPassword123!
    ulimits:
      memlock:
        soft: -1
        hard: -1
```

```
    nofile:
      soft: 65536
      hard: 65536
  volumes:
    - wazuh-indexer-data:/var/lib/wazuh-indexer
  networks:
    - wazuh-net


wazuh-manager:
  image: wazuh/wazuh-manager:4.7.3
  container_name: wazuh-manager
  restart: always
  ports:
    - "1514:1514/udp"
    - "1515:1515"
    - "514:514/udp"
    - "55000:55000"
  environment:
    - INDEXER_URL=https://wazuh-indexer:9200
    - INDEXER_USERNAME=admin
    - INDEXER_PASSWORD=SecretPassword123!
    - FILEBEAT_SSL_VERIFICATION_MODE=none
  volumes:
    - wazuh-manager-data:/var/ossec/data
    - wazuh-manager-logs:/var/ossec/logs
    - wazuh-manager-etc:/var/ossec/etc
  depends_on:
    - wazuh-indexer
  networks:
    - wazuh-net


wazuh-dashboard:
  image: wazuh/wazuh-dashboard:4.7.3
  container_name: wazuh-dashboard
  restart: always
```

```
    ports:
      - "443:5601"
    environment:
      - INDEXER_USERNAME=admin
      - INDEXER_PASSWORD=SecretPassword123!
      - WAZUH_API_URL=https://wazuh-manager
      - API_USERNAME=wazuh-wui
      - API_PASSWORD=MyS3cr37P450r.*-
    depends_on:
      - wazuh-indexer
      - wazuh-manager
    networks:
      - wazuh-net


volumes:
  wazuh-indexer-data:
  wazuh-manager-data:
  wazuh-manager-logs:
  wazuh-manager-etc:


networks:
  wazuh-net:
    driver: bridge
```

2.1.3. Portainer

Fichier : ~/cyberbox/portainer/docker-compose.yml

Portainer Community Edition fournit une interface graphique pour administrer les conteneurs Docker. Il monte le socket Docker pour un accès complet à l'environnement.

```
version: '3.8'


services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: always
    ports:
      - "9443:9443"
      - "9000:9000"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer-data:/data
    networks:
      - portainer-net


volumes:
  portainer-data:


networks:
  portainer-net:
    driver: bridge
```

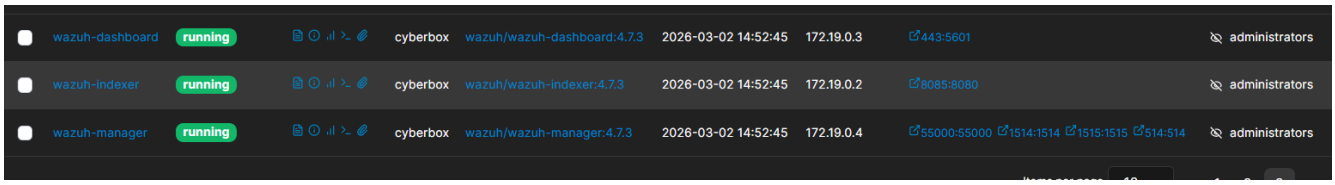
2.2 Portainer — Gestion Docker

Portainer fournit une interface web pour gérer l'ensemble des containers Docker de la plateforme.

Paramètre	Valeur
Port	9443 (HTTPS)
Accès	https://localhost:9443
Image	portainer/portainer-ce

2.3 Stack Wazuh — SIEM Principal

La stack Wazuh est composée de 3 containers interdépendants.

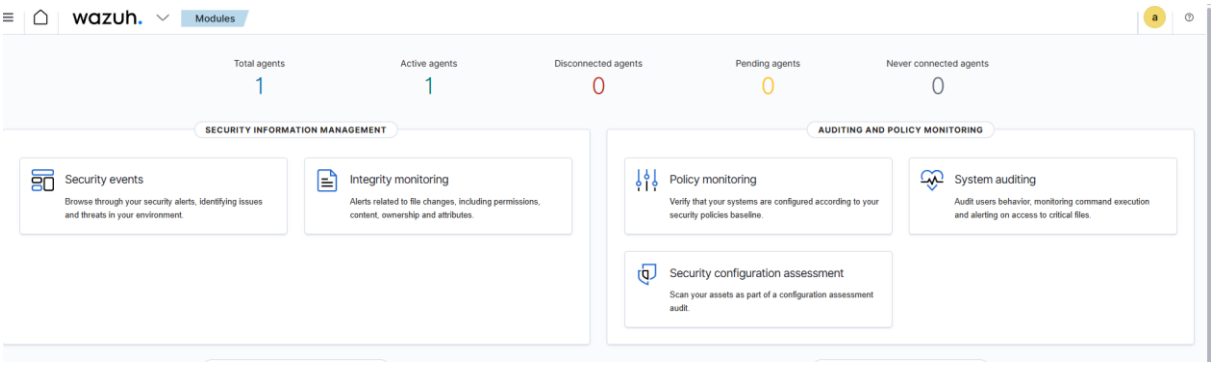


2.3.1wazuh-indexer (OpenSearch)

- Rôle : Base de données et moteur de recherche pour les alertes
- Port : 9200 (HTTPS)
- Image : wazuh/wazuh-indexer:4.7.3
- Credentials : admin / SecretPassword123!

2.3.2wazuh-manager

- Rôle : Cœur du SIEM, analyse les logs et génère les alertes
- Ports : 1514/udp (agents), 1515/tcp (enrollment), 55000/tcp (API REST), 514/udp (syslog)
- Image : wazuh/wazuh-manager:4.7.3
- Filebeat intégré : envoie les alertes vers l'indexer

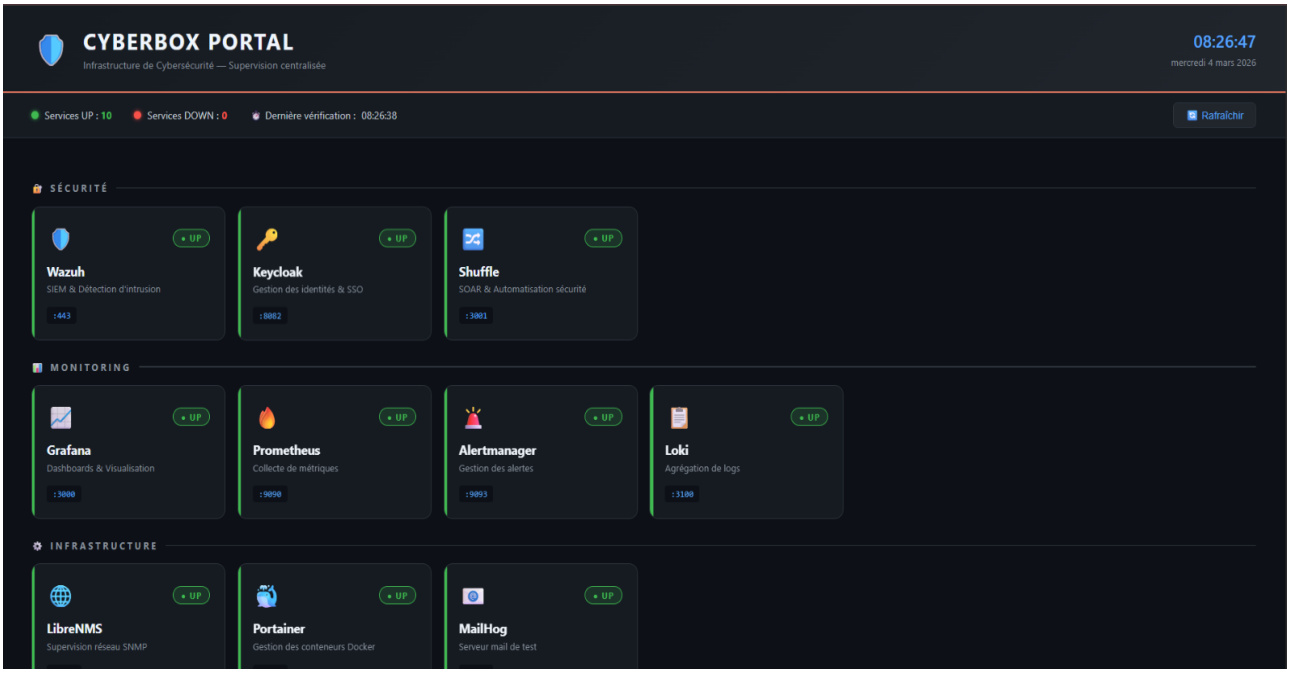


2.3.3 wazuh-dashboard (OpenSearch Dashboards)

- Rôle : Interface web de visualisation
- Port : 443 (HTTPS via Nginx)
- Image : wazuh/wazuh-dashboard:4.7.3

2.4 Nginx — Reverse Proxy

- Rôle : Point d'entrée unique, redirige vers les services
- Ports : 80 (HTTP → HTTPS) et 443 (HTTPS)
- Configuration : SSL avec certificats auto-signés



3. Configuration Réseau

3.1 Réseau Docker

Container	Réseau	IP
wazuh-indexer	wazuh_network (172.19.0.0/16)	172.19.0.x
wazuh-manager	wazuh_network (172.19.0.0/16)	172.19.0.6
wazuh-dashboard	wazuh_network (172.19.0.0/16)	172.19.0.x

3.2 Ports Exposés sur l'Hôte

Service	Port Hôte	Protocole	Usage
Nginx / Dashboard	443	HTTPS	Interface web principale
Nginx	80	HTTP	Redirect vers 443
Wazuh Indexer	9200	HTTPS	API OpenSearch

Service	Port Hôte	Protocole	Usage
Wazuh Manager	1514	UDP	Agents Wazuh
Wazuh Manager	1515	TCP	Enrollment agents
Wazuh Manager	55000	TCP	API REST Wazuh
Wazuh Manager	514	UDP	Syslog
Portainer	9443	HTTPS	Gestion Docker

4. Credentials Configurés

Service	Utilisateur	Mot de passe
Wazuh Dashboard	admin	SecretPassword123!
Wazuh Indexer	admin	SecretPassword123!
Wazuh API	wazuh-wui	(configuré dans docker-compose)
Portainer	admin	(défini au premier login)

5. Problèmes Rencontrés et Solutions

Problème 1 — vm.max_map_count trop bas

Cause : OpenSearch nécessite un vm.max_map_count suffisamment élevé pour fonctionner.

```
sysctl -w vm.max_map_count=262144
echo "vm.max_map_count=262144" >> /etc/sysctl.conf
```

Problème 2 — Certificats SSL manquants

Cause : Les certificats Wazuh n'étaient pas générés au démarrage.

```
docker run --rm \
  -v /root/cyberbox/wazuh/config/wazuh_indexer_ssl_certs:/certs \
  wazuh/wazuh-manager:4.7.3 \
  /var/ossec/bin/wazuh-certs-tool.sh -A
```

Problème 3 — Filebeat SSL verification_mode: full

Cause : Filebeat refusait les certificats auto-signés de l'indexer.

```
docker exec wazuh-manager sed -i \
  "s/ssl.verification_mode: 'full'/ssl.verification_mode: 'none'/" \
  /etc/filebeat/filebeat.yml
```

Problème 4 — Filebeat s'arrêtait après pkill

Cause : s6 (gestionnaire de processus du container) le redémarrait lentement.

```
sleep 15
docker exec wazuh-manager ps aux | grep filebeat
```

Problème 5 — Mémoire insuffisante

Cause : OpenSearch nécessite un minimum de 4 GB de RAM. Limitation JVM appliquée si nécessaire.

```
free -h
-Xms1g
-Xmx1g
```

Problème 6 — Index pattern déjà existant

Cause : Tentative de recréer wazuh-alerts-* déjà présent (version 9).

Erreur 409 Conflict → Normal, l'index pattern existait déjà. Aucune action requise.

6. Flux de Données

Le flux de données au sein de CyberBox suit le cheminement suivant :

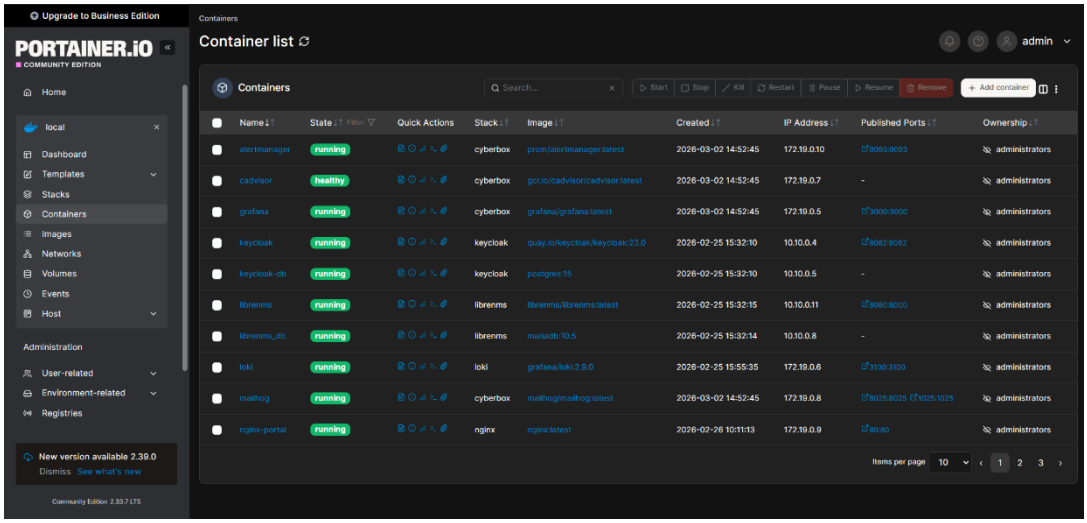
Étape	Composant	Description
1	Agents / Logs	Sources de données entrantes
2	wazuh-manager	Analyse des logs et application des règles
3	Filebeat	Transmission des alertes (intégré dans le manager)

Étape	Composant	Description
4	wazuh-indexer (HTTPS :9200)	Stockage dans OpenSearch
5	wazuh-dashboard	Visualisation des données
6	Nginx (:443)	Reverse proxy — point d'entrée
7	Navigateur utilisateur	Consultation de l'interface

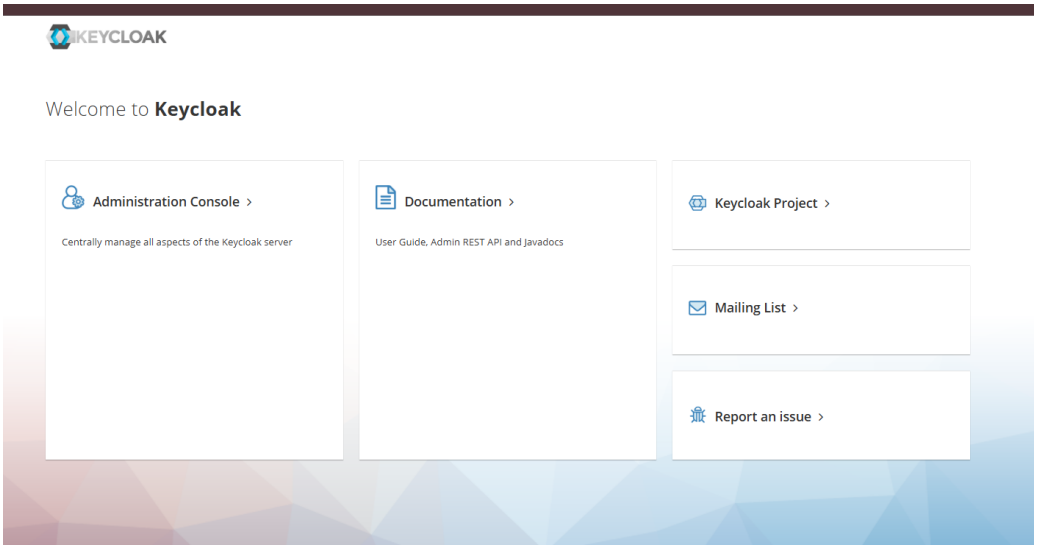
7. État Final Vérifié

Au dernier contrôle effectué, tous les composants étaient opérationnels :

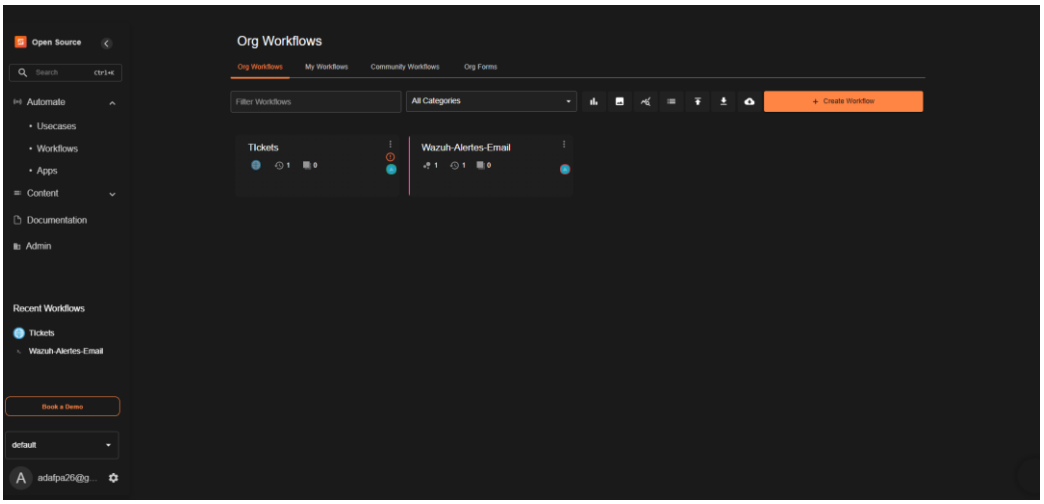
- ✅ wazuh-indexer — UP, cluster GREEN, 100% healthy
- ✅ wazuh-manager — UP, Filebeat actif (PID 1301)
- ✅ wazuh-dashboard — UP, accessible via HTTPS
- ✅ portainer — UP



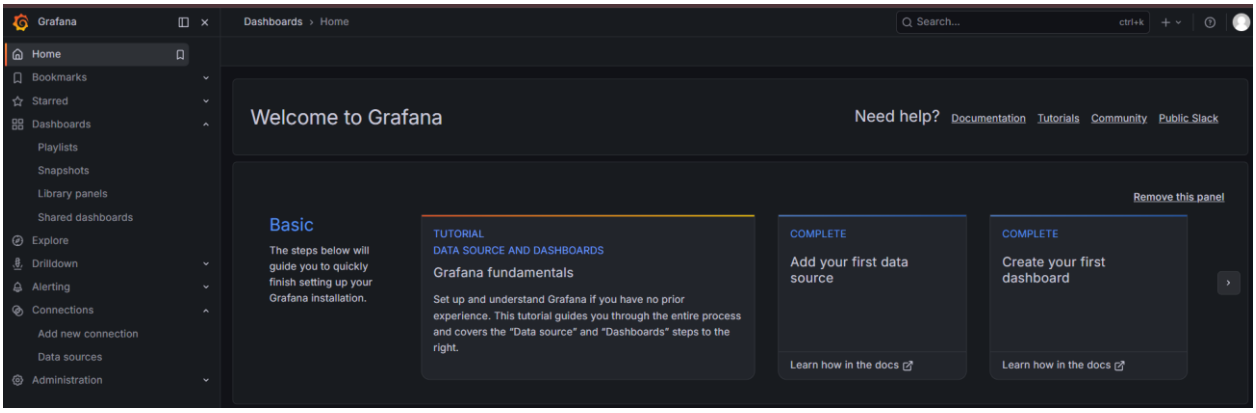
✅ KeyCLoak — UP / accessible en HTTPS :



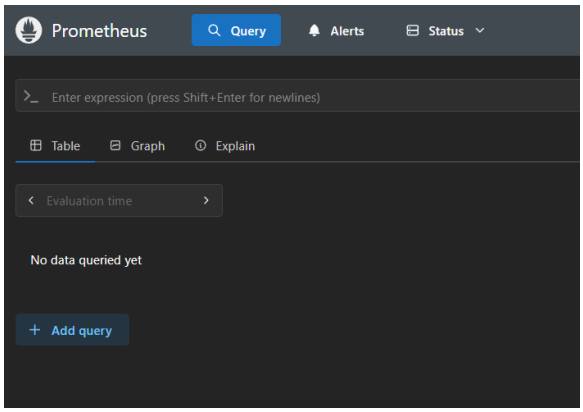
✅ Shuffle — UP/ accessible en HTTPS :



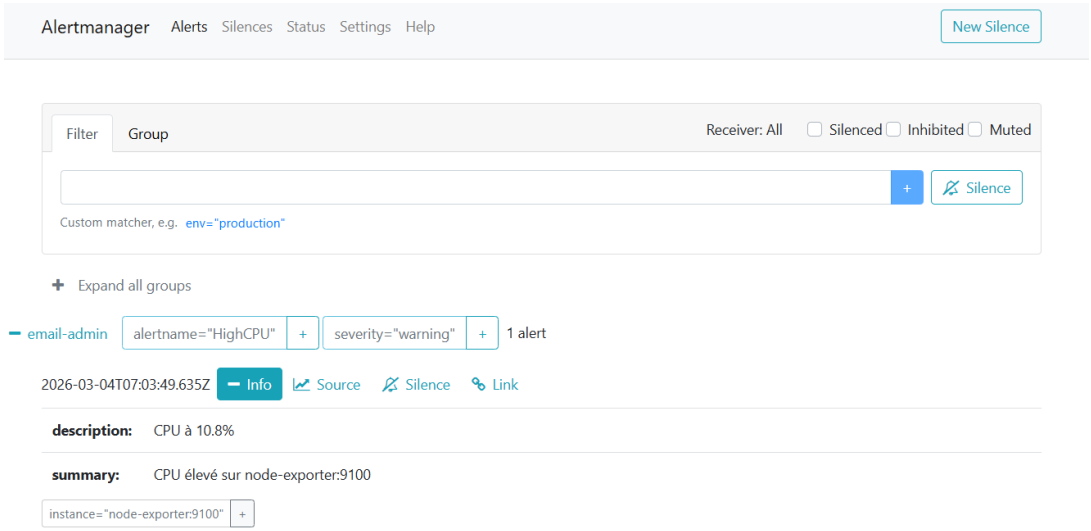
✅ Grafana — UP/ accessible en HTTPS :



✅ Prometheus — UP/ accessible en HTTPS :



✅ Alertmanager — UP/ accessible en HTTPS :



8. Commandes Utiles de Maintenance

8.1 Supervision

```
# État de tous les containers
docker ps
```

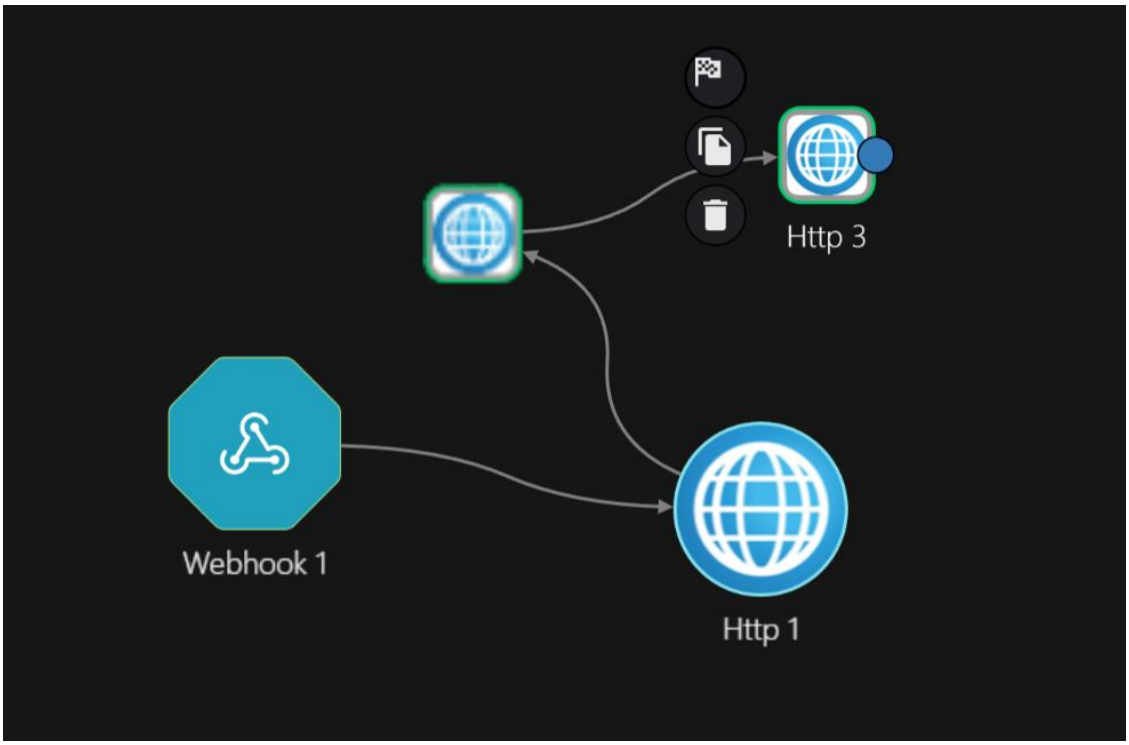
```
p, [::]:443->5601/tcp                                                
```

9. Intégration Wazuh → Shuffle → GLPI

9.1 Architecture du Workflow

Le pipeline d'automatisation suit les étapes suivantes :

Étape	Composant	Action
1	Wazuh	Génère une alerte sécurité
2	Shuffle (Webhook)	Réception de l'alerte JSON
3	Http 1 → GLPI	initSession — Authentification
4	Http 2 → GLPI	POST /Ticket — Création ticket
5	Http 3 → GLPI	killSession — Fermeture session
6	GLPI	Ticket créé automatiquement ☑



9.2 Configuration Wazuh (ossec.conf)

```
<!-- /var/ossec/etc/ossec.conf -->
<integration>
  <name>shuffle</name>
  <hook_url>http://[IP_SHUFFLE]:3001/api/v1/hooks/[WEBHOOK_ID]</hook_url>
  <level>3</level>
  <alert_format>json</alert_format>
</integration>

systemctl restart wazuh-manager
```

9.3 Workflow Shuffle — Détail des étapes

Trigger Webhook

```
Type : Webhook
→ Reçoit les alertes Wazuh en JSON
→ Variables :
   $exec.title → Titre alerte
   $exec.text  → Contenu alerte
```

Http 1 — initSession

```
Method : GET
URL    : http://192.168.254.85/apirest.php/initSession

Headers :
  Content-Type  : application/json
  App-Token     : 9GEjVJ39fCVQ8Hr...
  Authorization : Basic [BASE64_user:pass]

Retourne : $http_1.body.session_token
```

Http 2 — Création Ticket

```
Method : POST
URL    : http://192.168.254.85/apirest.php/Ticket

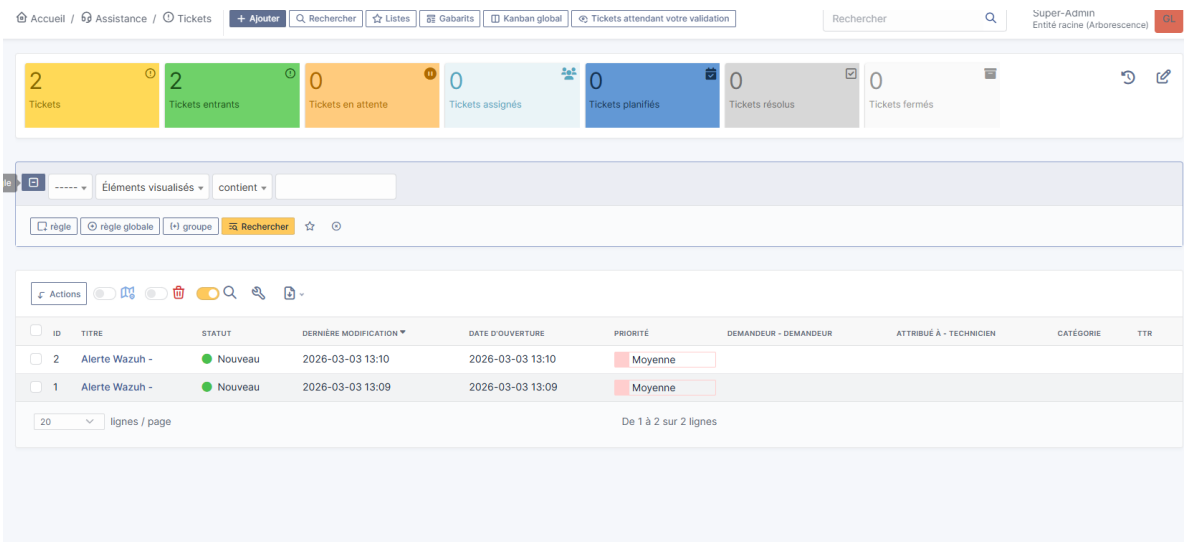
Body :
{
  "input": {
    "name"              : "Alerte Wazuh - $exec.title",
    "content"           : "$exec.text",
    "urgency"           : 3,
    "priority"          : 3,
    "itilcategories_id" : 1
  }
}

Résultat : status 201 ☑  |  body.id: 3  |  success: true ☑
```

Http 3 — killSession

```
Method : GET
URL    : http://192.168.254.85/apirest.php/killSession

Résultat : status 200 ☑
```



10. Supervision Avancée — Stack Monitoring (Docker)

10.1 Architecture de la Stack

La stack de monitoring s'articule autour de Prometheus comme collecteur central, alimenté par node-exporter et cAdvisor, avec Grafana pour la visualisation et Alertmanager pour la gestion des alertes.

Service	Rôle	Port
Prometheus	Collecte métriques	9090
Grafana	Visualisation	3000
Alertmanager	Gestion alertes	9093
Node-exporter	Métriques système	9100
cAdvisor	Métriques Docker	8080
Mailhog	SMTP de test	1025/8025
Loki	Agrégation logs	3100
Promtail	Collecte logs	9080

10.2 Configuration Prometheus

prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']

rule_files:
  - /etc/prometheus/rules/*.yml

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

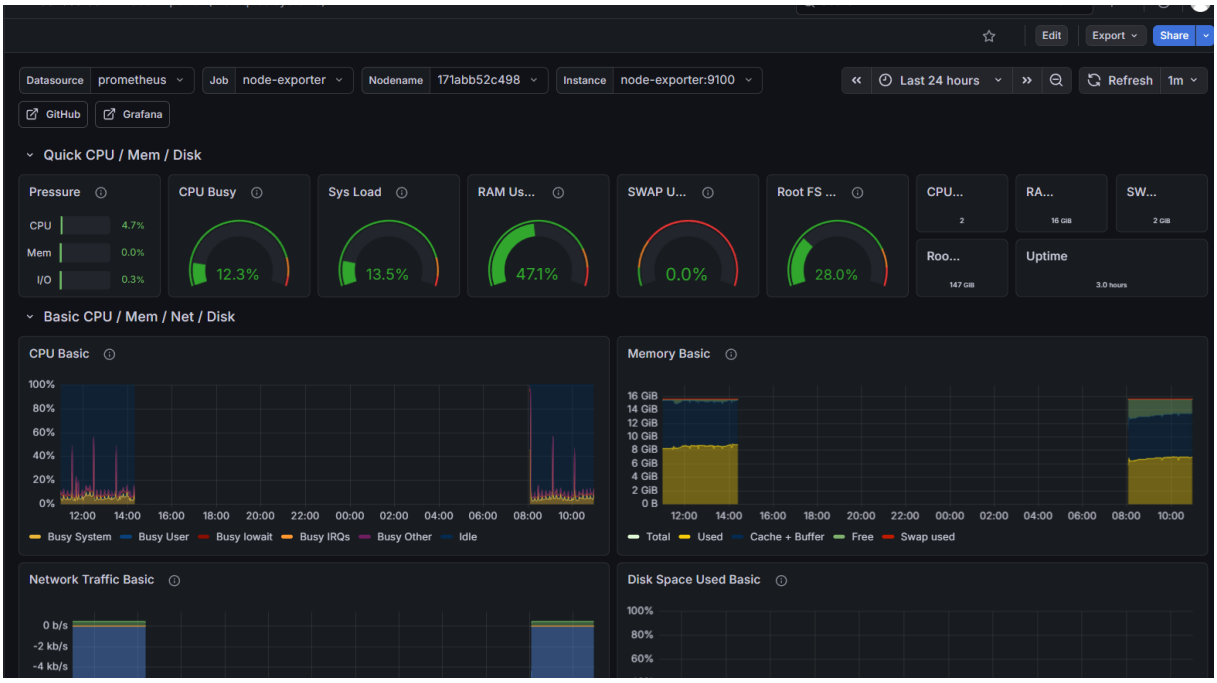
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

10.3 Connexion Prometheus → Grafana

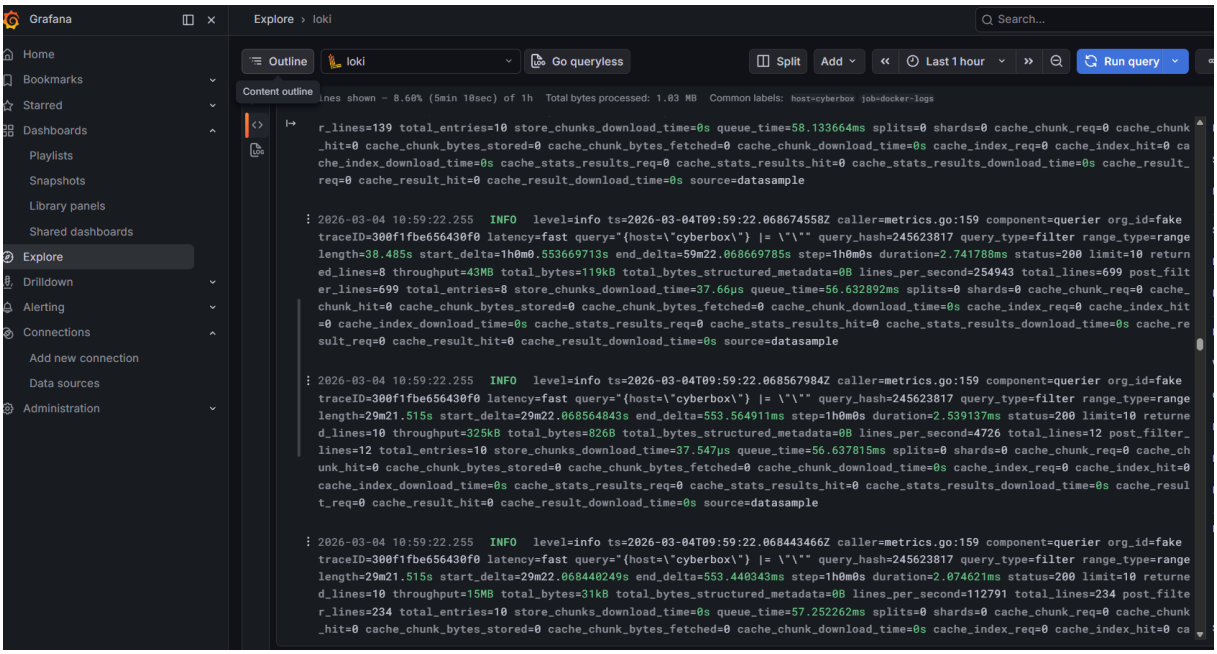
Ajout de la datasource dans Grafana

```
Grafana → Configuration → Data Sources
→ Add Data Source
→ Prometheus
→ URL : http://prometheus:9090
→ Save & Test ☑
```



Dashboards importés

- ☑ Node Exporter Full → ID : 1860 (CPU, RAM, Disque, Réseau)
- ☑ Docker cAdvisor → ID : 14282 (Métriques conteneurs)
- ☑ Loki Logs → ID : 13639 (Visualisation des logs)



Flux de données validés

```
node-exporter ──métriques──▶ Prometheus ──▶ Grafana ☑
cadvisor ─────────métriques──▶ Prometheus ──▶ Grafana ☑
Promtail ──────────────logs──▶ Loki ──────▶ Grafana ☑
```

10.4 Connexion Prometheus → Alertmanager

alertmanager.yml

```
global:
  smtp_smarthost: 'mailhog:1025'
  smtp_from: 'alertmanager@cyberbox.local'
  smtp_require_tls: false

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'email-alert'

receivers:
  - name: 'email-alert'
    email_configs:
      - to: 'admin@cyberbox.local'
        send_resolved: true
```

rules/alerts.yml

```
groups:
  - name: system_alerts
    rules:

      # Alerte si instance down
      - alert: InstanceDown
        expr: up == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Instance {{ $labels.instance }} down"
```

```
          description: "L'instance est hors ligne depuis plus d'1 minute"

      # Alerte CPU élevé
      - alert: HighCPUUsage
        expr: 100 - (avg
by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 5m
        labels:
          severity: warning

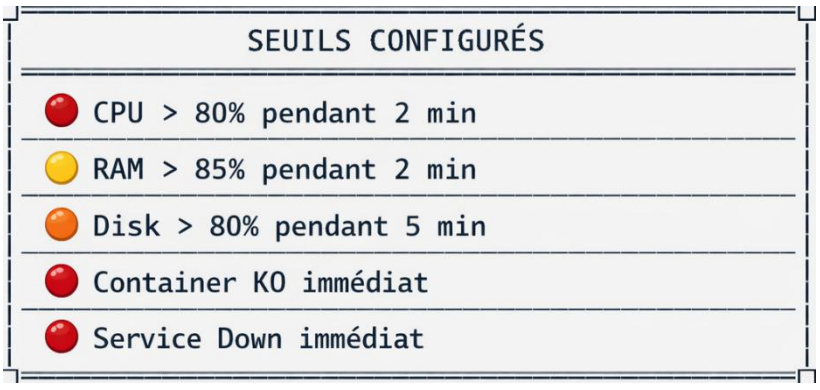
      # Alerte RAM élevée
      - alert: HighMemoryUsage
        expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
              / node_memory_MemTotal_bytes * 100 > 85
        for: 5m
        labels:
          severity: warning

      # Alerte disque plein
      - alert: DiskSpaceLow
        expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 15
        for: 5m
        labels:
          severity: critical
```

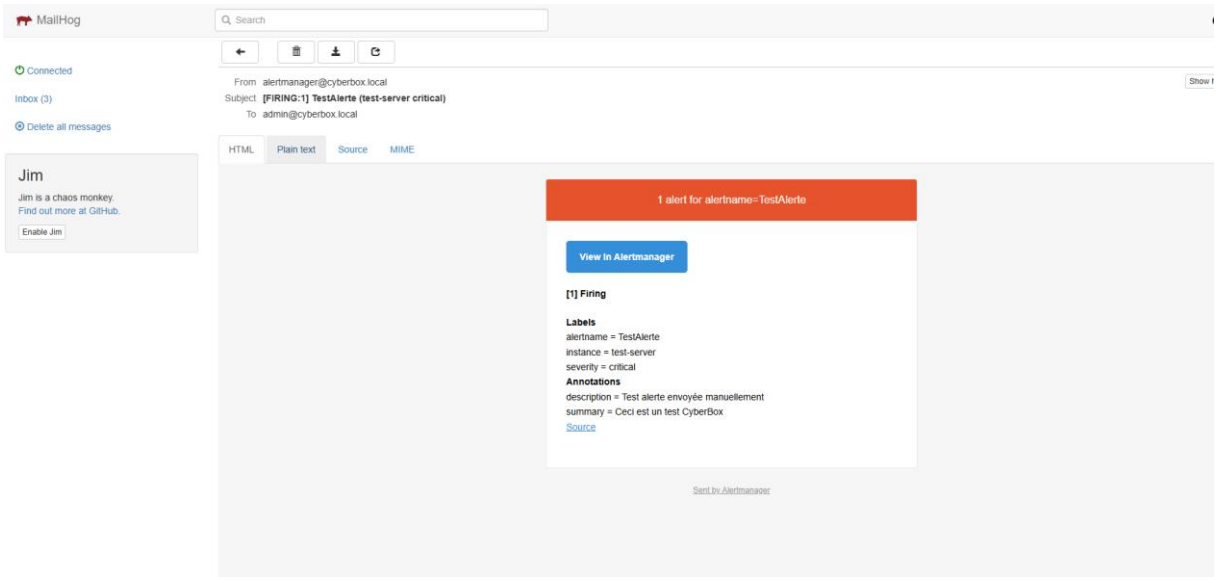
10.5 Résultats Obtenus

Prométheus ——————▶ Alertmanager ——————▶ Mailhog

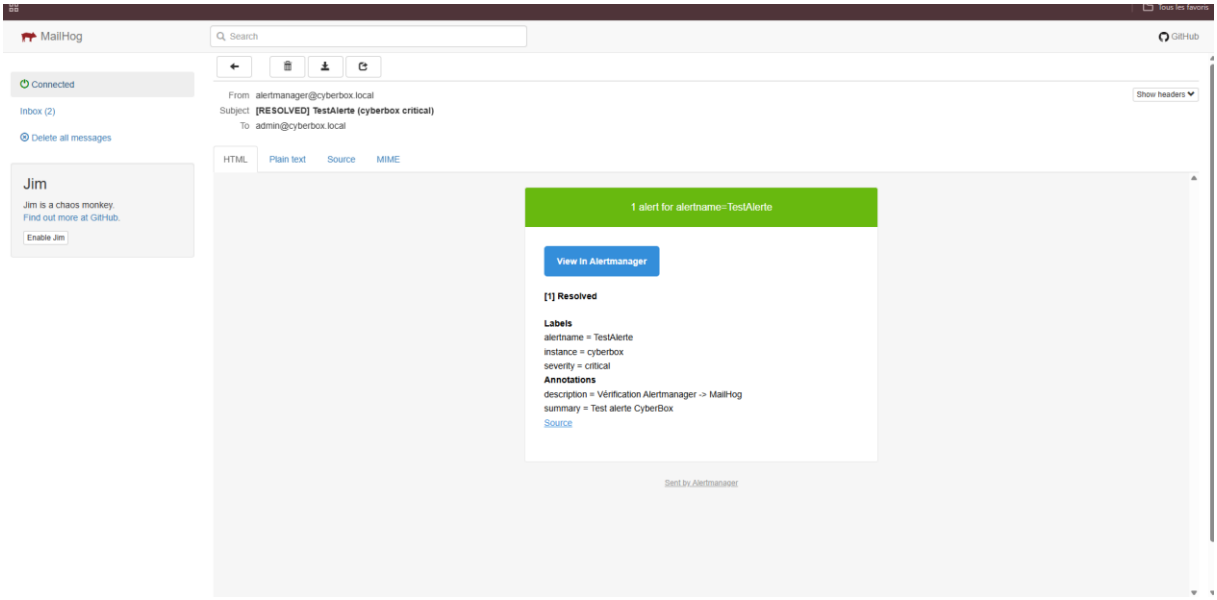
Configuration sur prométheus :



Alerte reçu par mail via alertmanager :

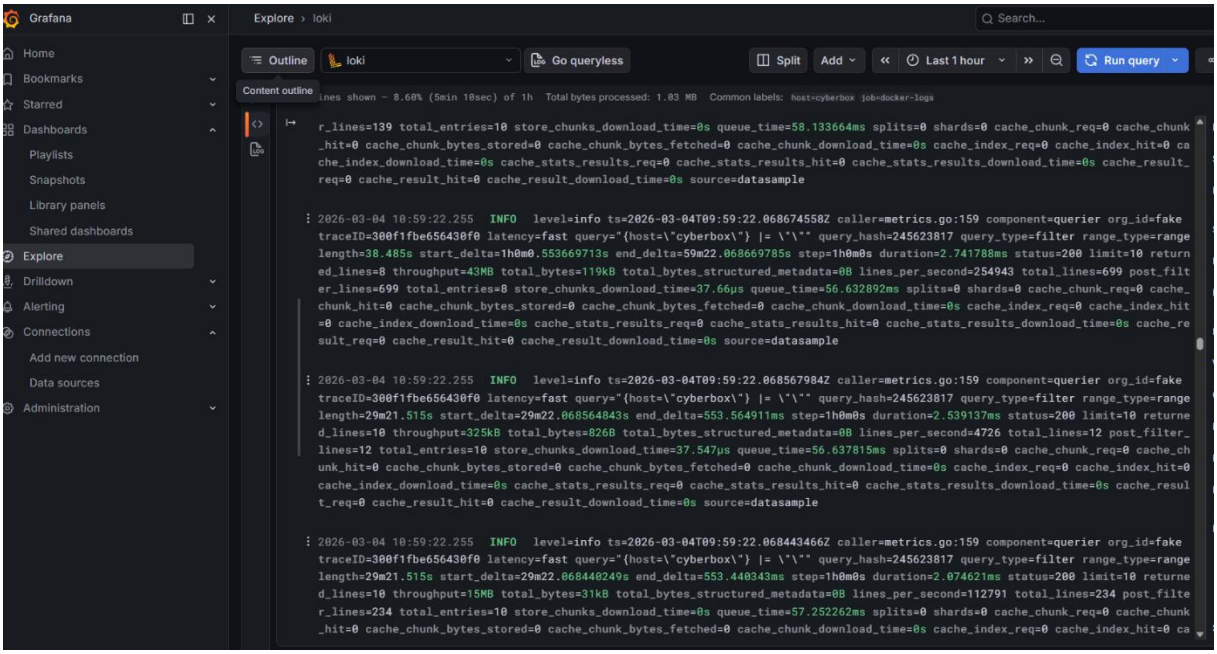


L'alerte est résolu :

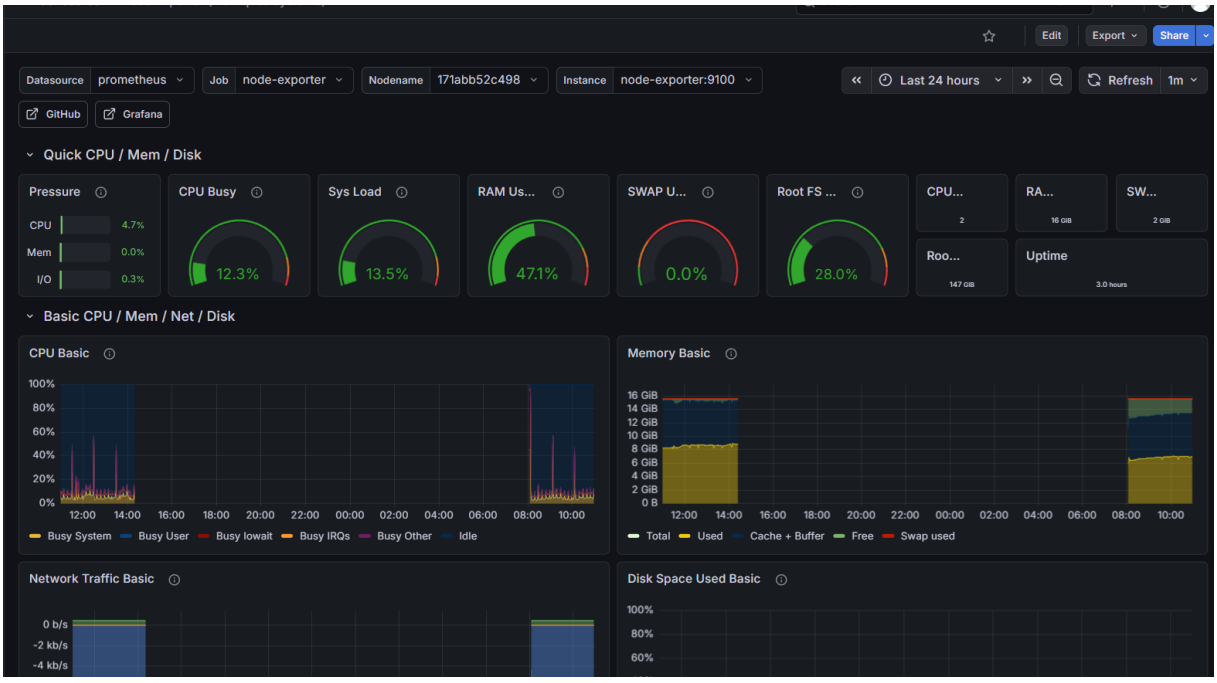


Promtail ——————▶ Loki ——————▶ Graphana

Promtail agit comme un agent, loki récupére les logs et Graphana demande les informations pour les afficher, centralisation des logs.



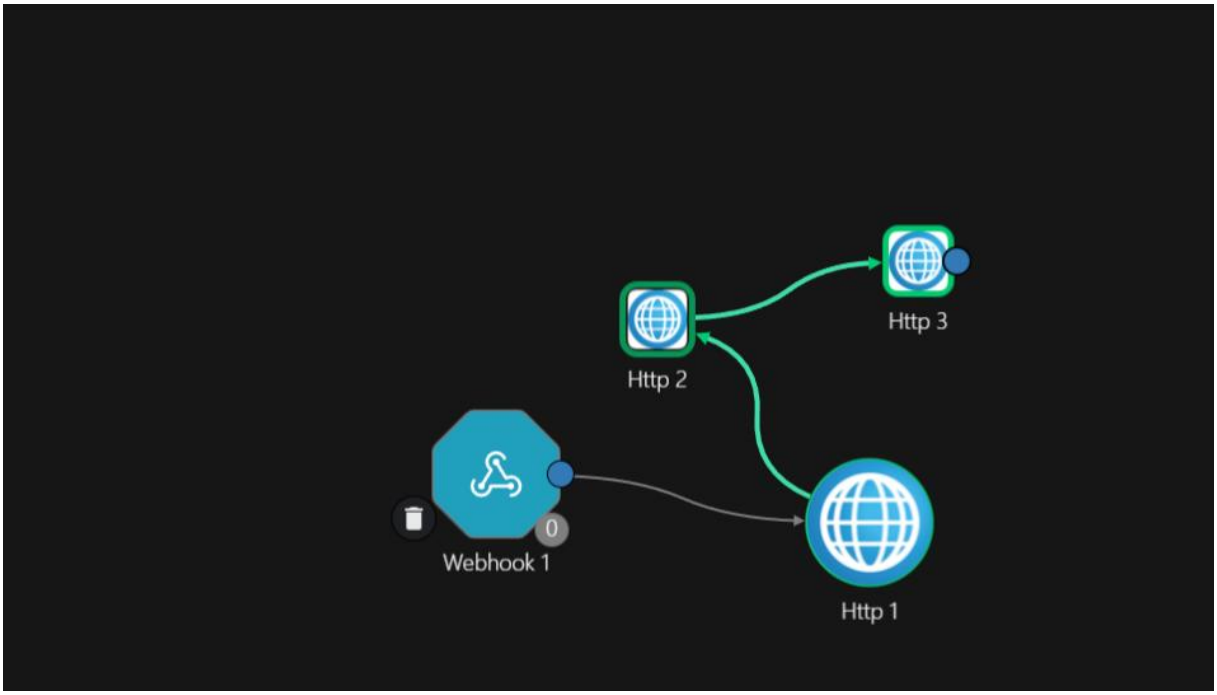
Node exporter – cAdvisor ——▶ Prométheus ——▶ Graphana



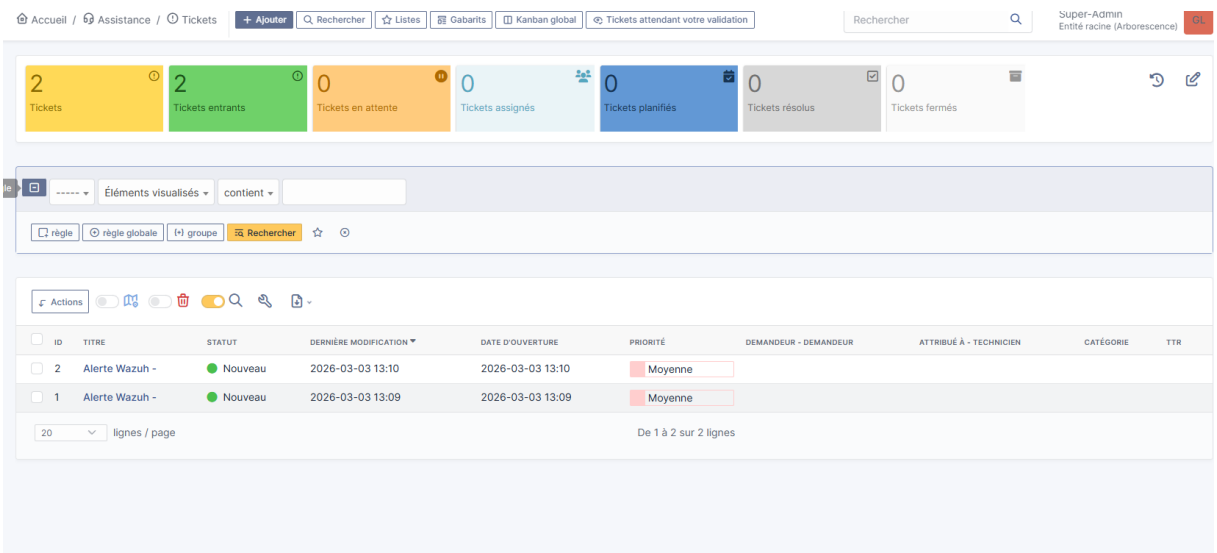
Wazuh ——————▶ Shuffle ——————▶ GLPI

Wazzuh récupére les logs par son agent,

Workflow Shuffle : création de tickets dans GLPI :



Wazuh envoie ses alertes vers Shuffle, et Shuffle va créer un ticket vers le GLPI par ce Flow :



VII - Glossaire

Acronyme / Terme	Définition
Ansible	Outil d'automatisation et de déploiement de configuration. Utilisé pour déployer tous les services via des **playbooks** (rôles docker, wazuh, librenms, keycloak, etc.).
API	*Application Programming Interface* — Interface permettant à deux logiciels de communiquer. Dans notre projet : le script Python appelle l'**API Proxmox** pour cloner la VM automatiquement.
CI	*Configuration Item* — Tout élément géré dans la CMDB : un serveur, un logiciel, une personne, un flux réseau. Chaque CI possède un ID unique (ex : M-001, A-006, PP-002).
CMDB	*Configuration Management Database* — Base de données recensant tous les actifs (CI) d'une infrastructure : matériels, logiciels, données et personnes. Dans notre projet, intégrée dans GLPI.
DAT	*Dossier d'Architecture Technique* — Document décrivant l'architecture complète d'une solution (schémas, plan d'adressage, flux réseau, choix techniques).
Docker	Plateforme de conteneurisation applicative. Tous les services du projet (Wazuh, LibreNMS, Keycloak, etc.) sont déployés via **Docker Compose**.
GLPI	*Gestionnaire Libre de Parc Informatique* — Outil ITSM open source utilisé pour la gestion des tickets d'incidents et l'hébergement de la CMDB.
IDS	*Intrusion Detection System* — Système de détection d'intrusion. Analyse le trafic réseau ou les logs système et **génère des alertes** en cas de comportement suspect, sans bloquer automatiquement. Dans notre projet, **Wazuh** joue partiellement ce rôle via ses règles de détection.
IPS	*Intrusion Prevention System* — Système de prévention d'intrusion. Contrairement à l'IDS, il ne se contente pas de détecter : il **bloque activement** le trafic malveillant en temps réel. Dans notre projet, le **Fortigate 100D** intègre des fonctionnalités IPS au niveau périmétrique.
ITIL	*Information Technology Infrastructure Library* — Ensemble de bonnes pratiques pour la gestion des services informatiques. Framework utilisé pour structurer notre projet (CMDB, gestion des incidents, des changements, etc.)
ITSM	*IT Service Management* — Gestion des services informatiques. Désigne les pratiques et outils (comme GLPI) permettant de gérer les incidents, demandes et actifs IT.
LXC	*Linux Container* — Conteneur système Linux, plus léger qu'une VM. Utilisé en option pour isoler certains services secondaires.
MFA	*Multi-Factor Authentication* — Authentification à plusieurs facteurs (mot de passe + code OTP par exemple). Obligatoire sur tous les services web via Keycloak.

NAT	*Network Address Translation* — Traduction d'adresses réseau. Permet aux équipements du LAN d'accéder à Internet via une adresse publique unique (configuré sur le **Fortigate**).
OIDC	*OpenID Connect* — Protocole d'authentification basé sur OAuth 2.0, utilisé par Keycloak pour protéger les services web (Wazuh, LibreNMS, Portainer...).
PCA / PCI	*Plan de Continuité d'Activité / Plan de Continuité Informatique* — Ensemble de procédures permettant de maintenir ou restaurer les services en cas d'incident majeur.
POC	*Proof of Concept* — Preuve de concept. Phase de test réalisée sur **VMware Workstation** avant déploiement en production sur Proxmox.
SIEM	*Security Information and Event Management* — Outil de centralisation et d'analyse des logs de sécurité. Dans notre projet : **Wazuh Manager**.
SMTP	*Simple Mail Transfer Protocol* — Protocole d'envoi d'e-mails. Dans notre projet : **msmtp ou Postfix** (relay-only) pour l'envoi des alertes mail (Wazuh, LibreNMS, Shuffle).
SNMP	*Simple Network Management Protocol* — Protocole de supervision réseau. Utilisé par **LibreNMS** pour collecter les métriques des switchs, routeurs et AP (en version SNMPv3).
SOAR	*Security Orchestration, Automation and Response* — Outil d'automatisation de la réponse aux incidents de sécurité. Dans notre projet : **Shuffle**.
SSO	*Single Sign-On* — Authentification unique permettant à un utilisateur de se connecter une seule fois pour accéder à tous les services. Dans notre projet : **Keycloak**.

VIII - Conclusion

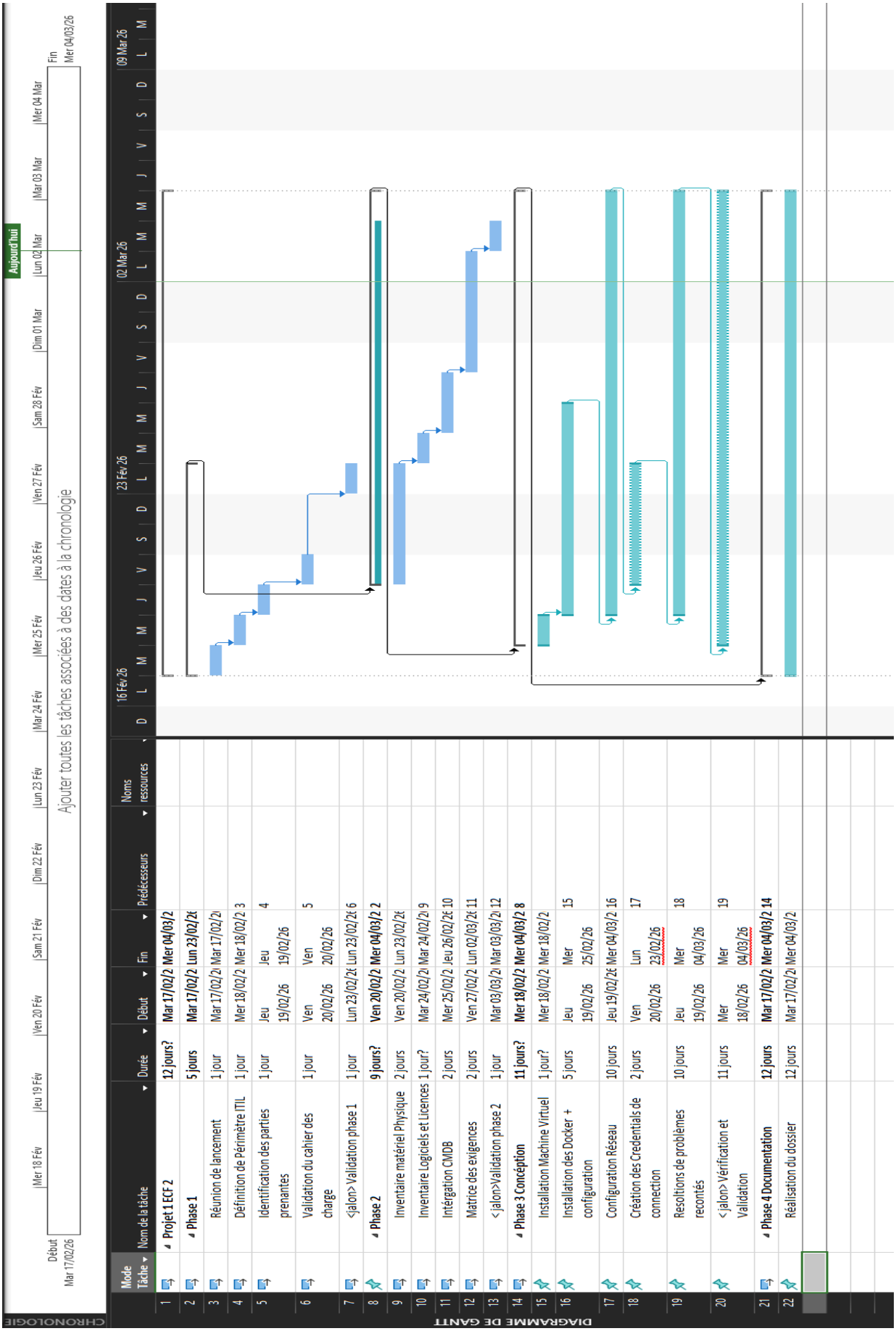
Ce projet de supervision d'infrastructure AIS représente une réponse complète et structurée aux enjeux de visibilité, de sécurité et d'automatisation d'un environnement informatique moderne. En s'appuyant sur une stack technique entièrement open source — Wazuh, Prometheus, Grafana, Loki, Shuffle, Keycloak et GLPI — déployée via Docker sur une VM Debian 12, nous avons construit une infrastructure de supervision cohérente, évolutive et conforme aux bonnes pratiques ITIL v4.

L'ensemble des exigences fonctionnelles et non fonctionnelles définies en début de projet ont été couvertes : détection des intrusions, supervision réseau, centralisation des logs, automatisation des workflows d'alerte, authentification unique et gestion des tickets. La mise en place du workflow Wazuh → Shuffle → GLPI illustre concrètement la capacité du système à réagir de manière autonome face à un incident de sécurité.

Ce projet a également permis à l'équipe de monter en compétences sur des technologies clés du domaine de la cybersécurité et de l'administration système, tout en appliquant une démarche de gestion de projet rigoureuse, avec un versionning documentaire, une CMDB structurée et un plan d'adressage réseau détaillé.

Il constitue une base solide et documentée, prête à être industrialisée sur un environnement ESXI en production.

Axe d'évolution	Description	Raison de non-réalisation
SCADA & Automate	Intégration d'un environnement SCADA avec supervision d'automates industriels dans un réseau OT sécurisé et isolé	Manque de temps et de moyens matériels pour mettre en place un cadre sécurisé adapté
Ansible	Automatisation complète du déploiement de la CyberBox via des playbooks Ansible (rôles docker, wazuh, loki, keycloak, etc.)	Manque de temps et de documentation au préalable sur le YAML.
LibreNMS	Supervision réseau complète des équipements physiques (switchs, routeurs, AP) via SNMPv3	Non déployé par manque de temps, l'environnement réseau physique complet n'étant pas disponible dans le cadre du POC
Keycloak & MFA	Mise en place du SSO avec authentification multi-facteurs obligatoire sur tous les services web (Wazuh, Portainer, GLPI...)	Non intégré faute de temps, la configuration OIDC et MFA nécessitant une infrastructure stable et complète au préalable


CHRONOLOGIE
Début
Mar 17/02/26
Ajouter toutes les tâches associées à des dates à la chronologie
Fin
Mer 04/03/26
Aujourd'hui
DIAGRAMME DE GANTT
Mode Tâche
Nom de la tâche
Durée
Début
Fin
Prédécesseurs
Noms ressources
1 Projet 1 ECF 2 12 jours? Mar 17/02/2 Mer 04/03/2
2 Phase 1 5 jours Mar 17/02/2 Lun 23/02/2
3 Réunion de lancement 1 jour Mar 17/02/2 Mar 17/02/2
4 Définition de Périmètre ITIL 1 jour Mer 18/02/2 Mer 18/02/2 3
5 Identification des parties prenantes 1 jour Jeu 19/02/26 Jeu 19/02/26 4
6 Validation du cahier des charge 1 jour Ven 20/02/26 Ven 20/02/26 5
7 <jalon> Validation phase 1 1 jour Lun 23/02/2 Lun 23/02/2 6
8 Phase 2 9 jours? Ven 20/02/2 Mer 04/03/2 2
9 Inventaire matériel Physique 2 jours Ven 20/02/2 Lun 23/02/2
10 Inventaire Logiciels et Licences 1 jour? Mar 24/02/2 Mar 24/02/2 9
11 Intégration CMDB 2 jours Mer 25/02/2 Jeu 26/02/2 10
12 Matrice des exigences 2 jours Ven 27/02/2 Lun 02/03/2 11
13 <jalon>Validation phase 2 1 jour Mar 03/03/2 Mar 03/03/2 12
14 Phase 3 Conception 11 jours? Mer 18/02/2 Mer 04/03/2 8
15 Installation Machine Virtuel 1 jour? Mer 18/02/2 Mer 18/02/2
16 Installation des Docker + configuration 5 jours Jeu 19/02/26 Mer 25/02/26 15
17 Configuration Réseau 10 jours Jeu 19/02/2 Mer 04/03/2 16
18 Création des Credentials de connection 2 jours Ven 20/02/26 Lun 23/02/26 17
19 Resoltions de problèmes recontés 10 jours Jeu 19/02/26 Mer 04/03/26 18
20 <jalon> Vérification et Validation 11 jours Mer 18/02/26 Mer 04/03/26 19
21 Phase 4 Documentation 12 jours Mar 17/02/2 Mer 04/03/2 14
22 Réalisation du dossier 12 jours Mar 17/02/2 Mer 04/03/2
16 Fév 26
23 Fév 26
02 Mar 26
09 Mar 26
Mer 18 Fév
Jeu 19 Fév
Ven 20 Fév
Sam 21 Fév
Dim 22 Fév
Lun 23 Fév
Mar 24 Fév
Mer 25 Fév
Jeu 26 Fév
Ven 27 Fév
Sam 28 Fév
Dim 01 Mar
Lun 02 Mar
Mar 03 Mar
Mer 04 Mar